Association analysis

Module 3

Association analysis

Association analysis is useful for discovering interesting relationship hidden in large data set.

Relationship can be represented in the form of association rules or sets of frequent items

Example

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Following rule can be extracted:

{Diapers}→ {Beer}

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Binary Representation

→ Market basket data can be represented in a binary format

TID	bread	Milk	Diapers	Beer	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

ItemSet

In association analysis, a collection of zero or more items is termed as - itemset.

For instance:

{Beer, Diapers, Milk} is an example of a 3-itemset.

The null (or empty) set is an itemset that does not contain any items.

Transaction Width: It is defined as the number of items present in a transaction.

Association rule

An association rule is an implication expression of the form $X \rightarrow Y$ where X and Y are disjoint itemset

i.e. $X \cap Y = \phi$

The strength of an association rule can be measured in terms of its **support** and **confidence**

$$\text{Support, } s(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{N};$$

$$\text{Confidence, } c(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}.$$

Support: The frequency of an item or itemset in the entire dataset.

How often an item or a set of items appears in the dataset.

Example: If "Milk" appears in 3 out of 10 transactions, the support for "Milk" is **3/10 = 0.3 (30%)**.

Confidence: The likelihood that item B is bought when item A is bought

Given that one item is present, how often does another item appear?

If 3 out of 5 transactions that include "Bread" also contain "Butter," then the confidence for the rule **"If Bread, then Butter"** is **3/5 = 0.6 (60%)**.

Consider the rule

{milk, diaphers}→{beer}

σ {milk,diapher,beer}=2

total no of transaction =5

Support, s(X→Y)

2/5

0.4

40%

Confidence, c(X→Y)

2/3

0.67

67%

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

TID	Items
100	A, B, C
200	A, C
300	A, D
400	B, E

A→C[Support=50%, confidence=66.6%]

C→A[Support=50%, confidence=100%]

Association rule discovery

Given a set of transactions T, the goal of association rule mining is to find all rules having

support ≥ minsup threshold

confidence ≥ minconf threshold

Apply the brute-force approach for mining association rules

Brute force approach for mining association rule is to compute **support** and **confidence** for every possible rules

This approach is expensive because there are exponentially many rules that can be extracted from a data set

The total number of possible rules extracted from a data set that contains D items is

$$R=3^{d}-2^{d+1}+1$$

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Compute the support and confidence for

$3^6 - 2^7 + 1 = 602$ rules.

More than 80% of the rules are discarded after applying minsup = 20% and minconf = 50%,

Association rule mining algorithm decompose the problem into two major approaches

- Frequent Itemset Generation
- Rule Generation

Frequent Itemset Generation, whose objective is to find frequent itemsets.

frequent itemset is an itemsets whose support is greater than or equal to minsup threshold

Rule Generation, whose objective is to extract rules that satisfy both minsup and minconf

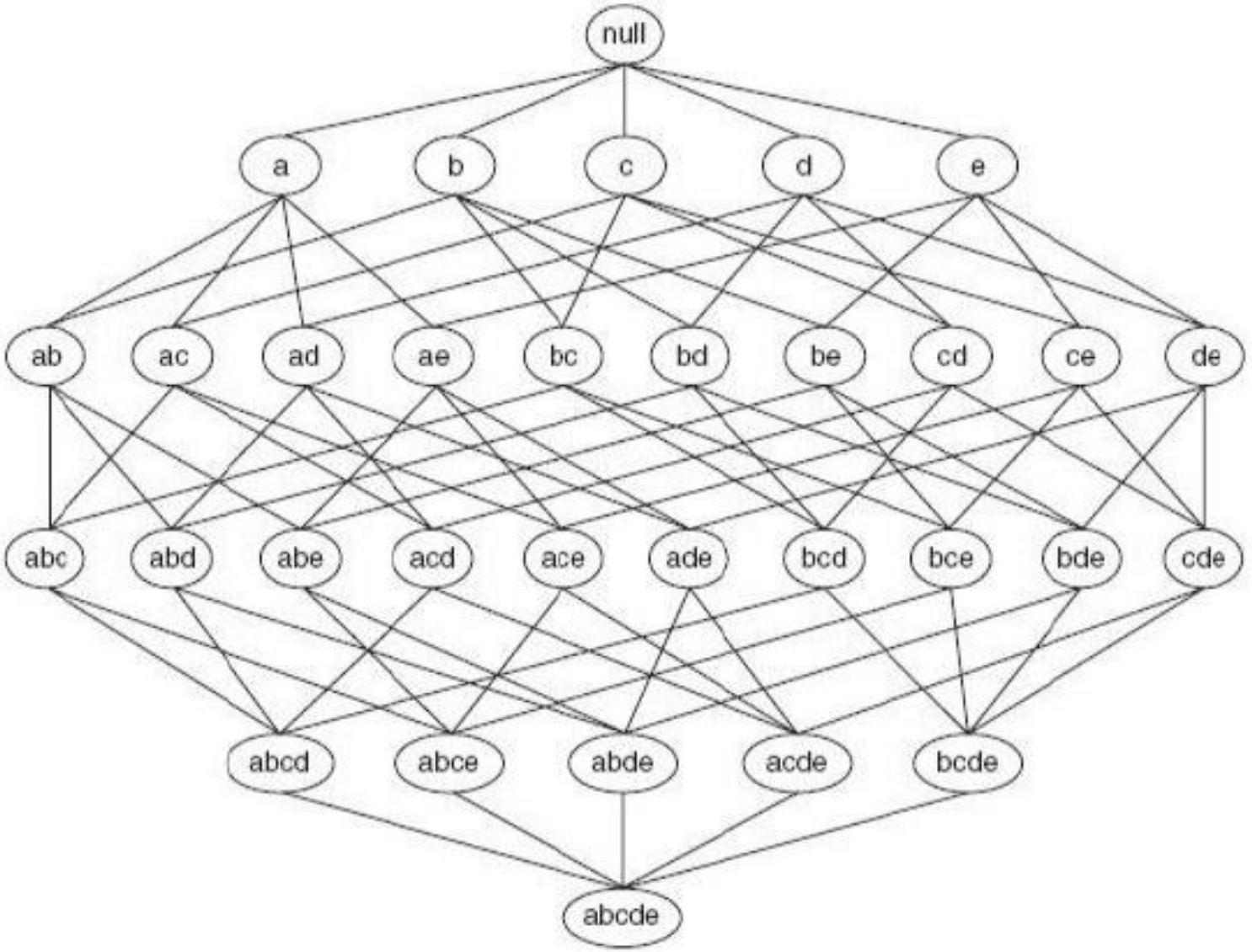
These rules are called strong rules

A lattice structure can be used to enumerate the list of all possible itemsets.

An itemset lattice for I = {a, b, c, d, e}. In general, a data set that contains k items can potentially generate up to 2^k - 1 frequent itemsets, excluding the null set.

Because k can be very large in many practical applications, the search space of itemsets that need to be explored is exponentially large.

A Itemset lattice structure



A brute-force approach for finding frequent itemsets is to determine the support count for every candidate itemset in the lattice structure.

For example, the support for {Bread,Milk} is incremented three times because the itemset is contained in transactions 1, 4, and 5. Such an approach can be very expensive

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

To reduce computational complexity of frequent itemset generation:

→Reduce the number of candidate itemset

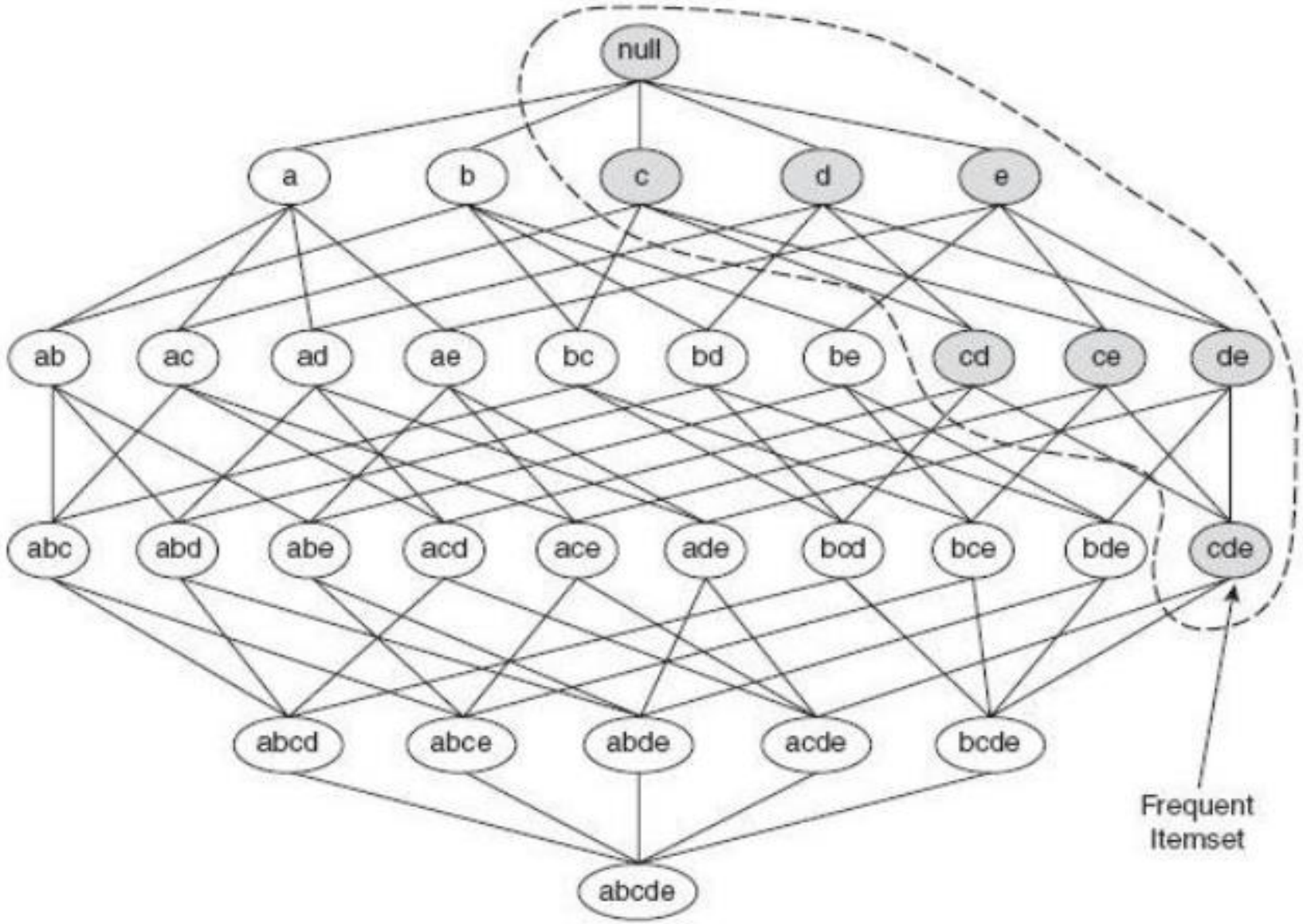
→Reduce the number of comparisons

The Apriori Principle

If an itemset is frequent, then all of its subsets must also be frequent.

Suppose {c, d, e} is a frequent itemset.

Clearly, all the subsets of {c, d, e} ie. {c, d},{c, e}, {d, e}, {c}, {d}, and {e} must also be frequent.


null
a
b
c
d
e
ab
ac
ad
ae
bc
bd
be
cd
ce
de
abc
abd
abe
acd
ace
ade
bcd
bce
bde
cde
abcd
abce
abde
acde
bcde
abcde
Frequent
Itemset

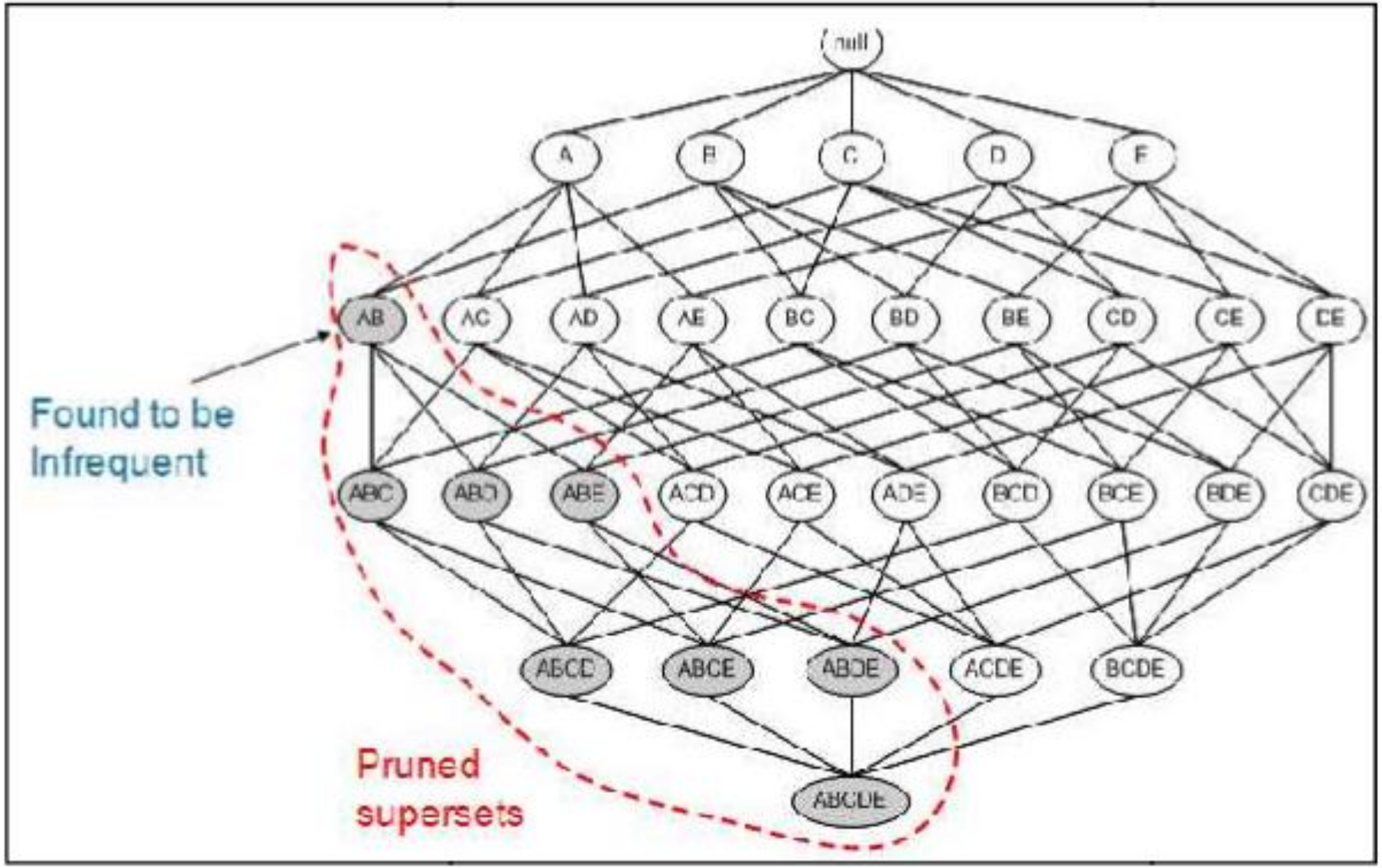


Figure 3.4: An illustration of the apriori principle for Infrequent Itemset

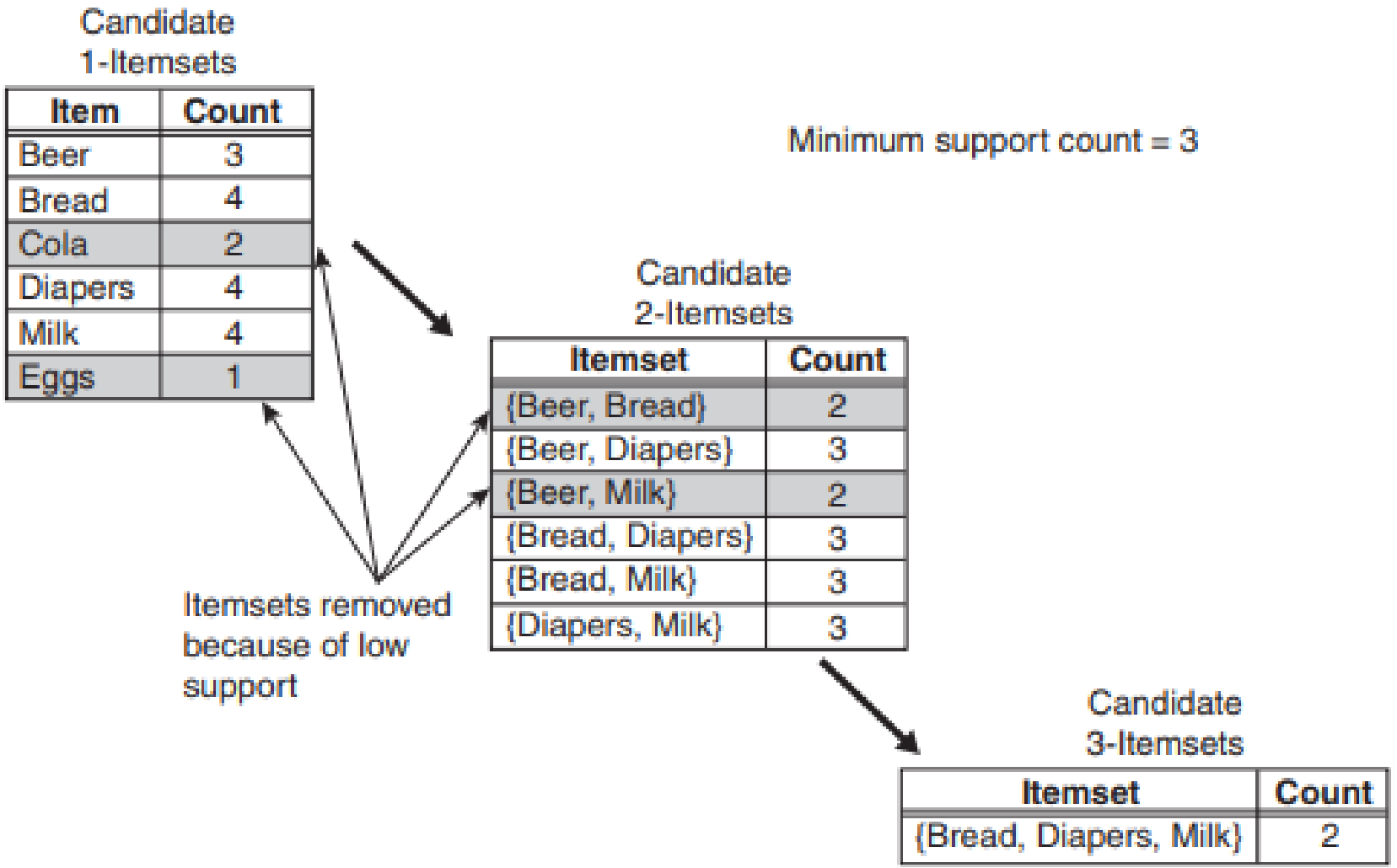
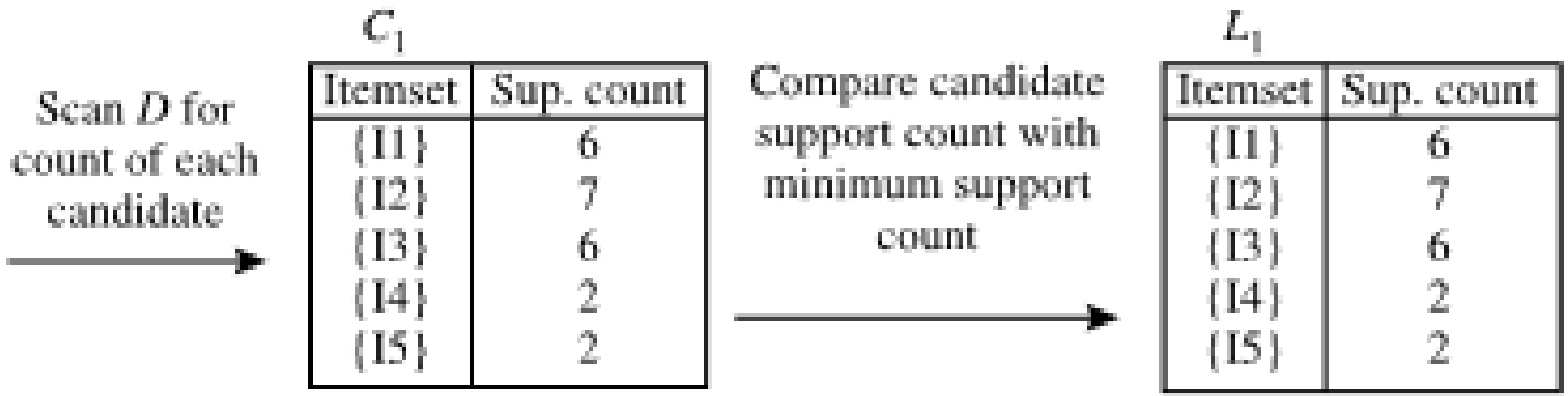


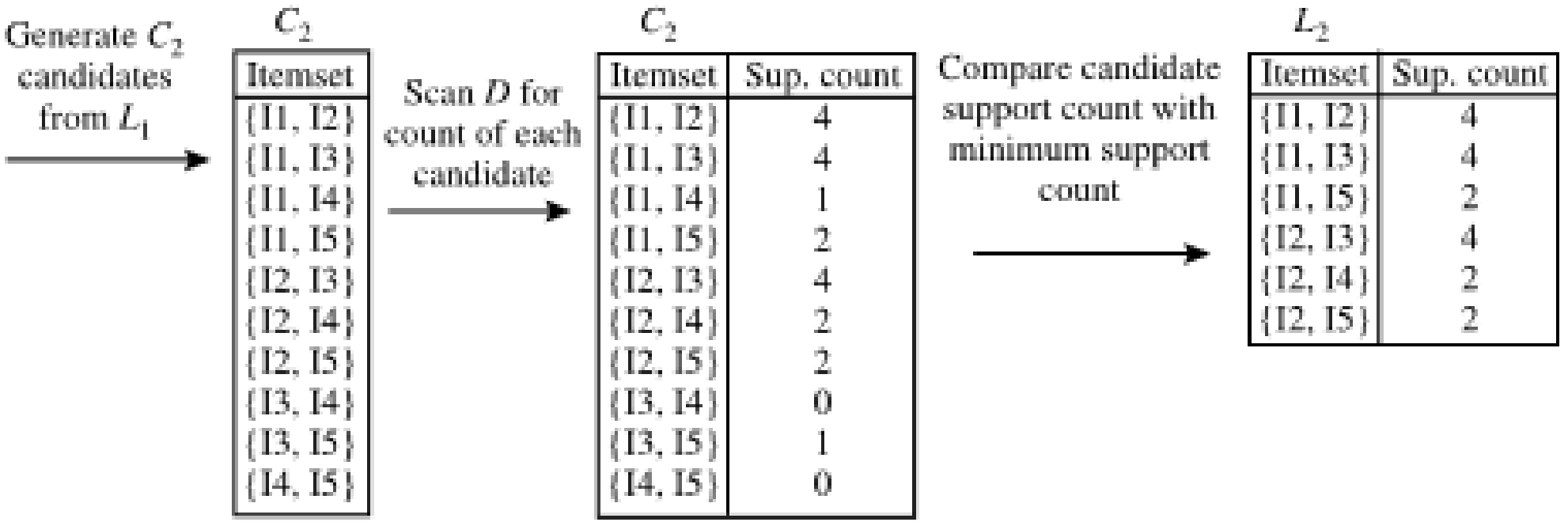
Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Table 6.1 Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3


Scan D for count of each candidate
C1
Itemset
Sup. count
{I1}
6
{I2}
7
{I3}
6
{I4}
2
{I5}
2
Compare candidate support count with minimum support count
L1
Itemset
Sup. count
{I1}
6
{I2}
7
{I3}
6
{I4}
2
{I5}
2


Generate C_2 candidates from L_1
C_2
Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}
Scan D for count of each candidate
C_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I4} 1
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2
{I3, I4} 0
{I3, I5} 1
{I4, I5} 0
Compare candidate support count with minimum support count
L_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2

I1, I2,I3

I1,I2,I5

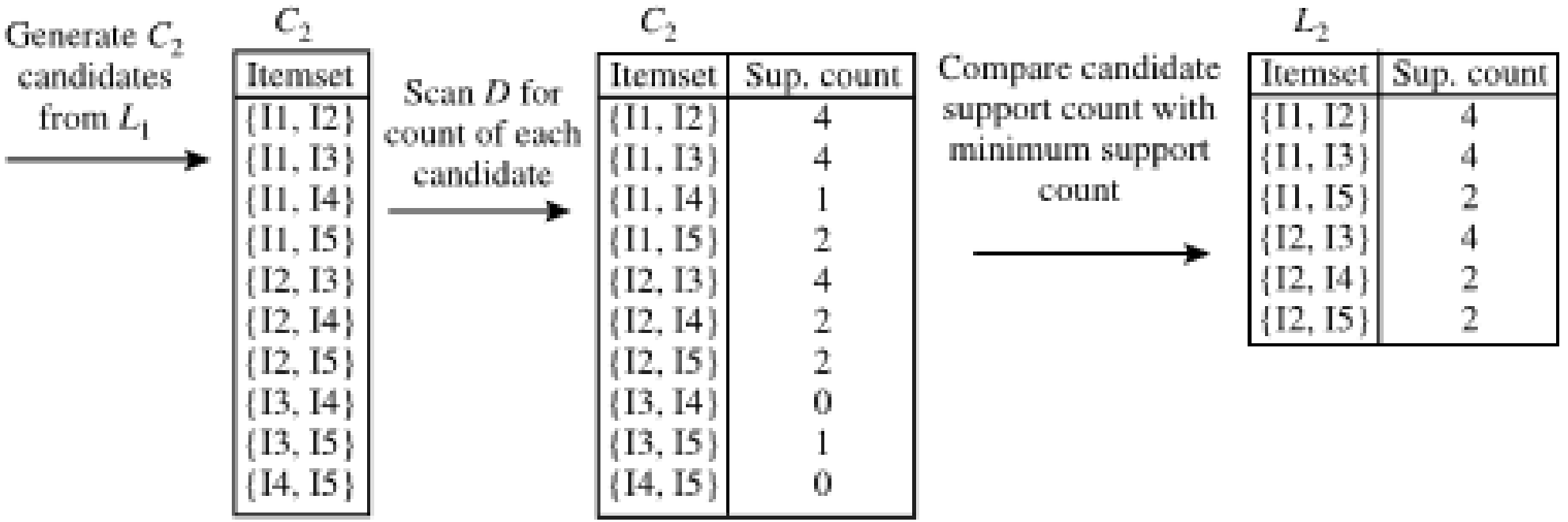
I1,I3,I5

I1, I2,I4

I2,I3,I4

I2,I3,I5

I2,I4,I5


Generate C_2 candidates from L_1
C_2
Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}
Scan D for count of each candidate
C_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I4} 1
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2
{I3, I4} 0
{I3, I5} 1
{I4, I5} 0
Compare candidate support count with minimum support count
L_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2

I1, I2,I3

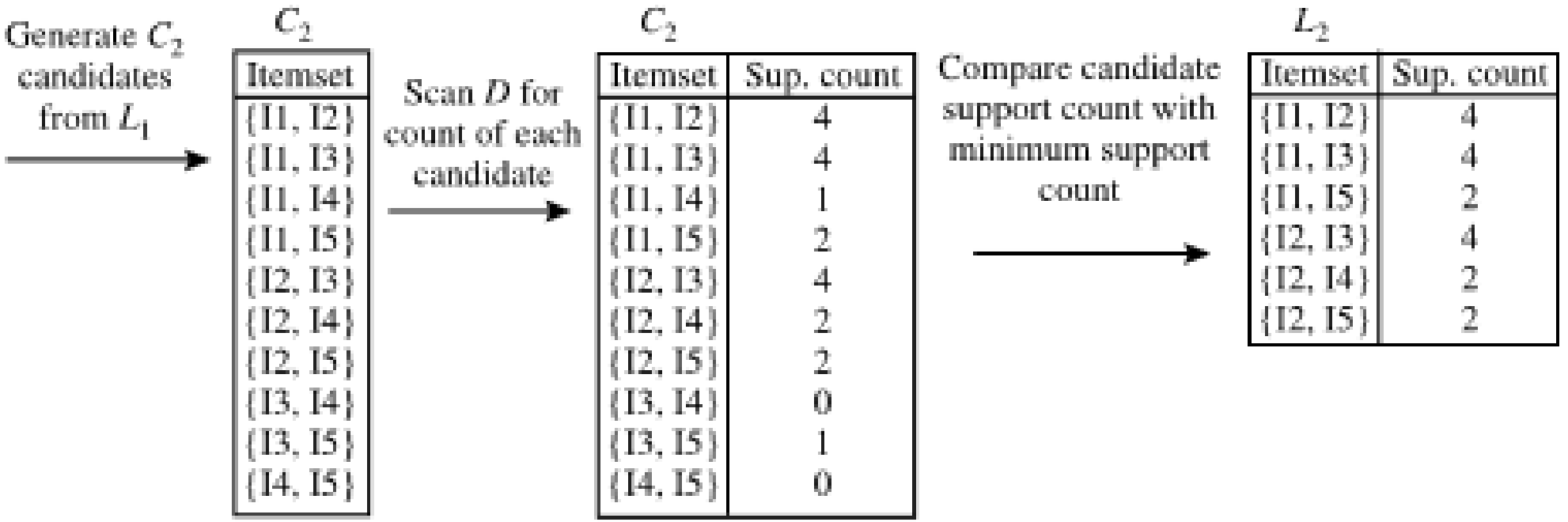
I1,I2,I5

I1,I3,I5

I2,I3,I4

I2,I3,I5

I2,I4,I5


Generate C_2 candidates from L_1
C_2
Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}
Scan D for count of each candidate
C_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I4} 1
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2
{I3, I4} 0
{I3, I5} 1
{I4, I5} 0
Compare candidate support count with minimum support count
L_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2

I1, I2,I3

I1,I2,I5

I1,I3,I5

I2,I3,I4

I2,I3,I5

I2,I4,I5

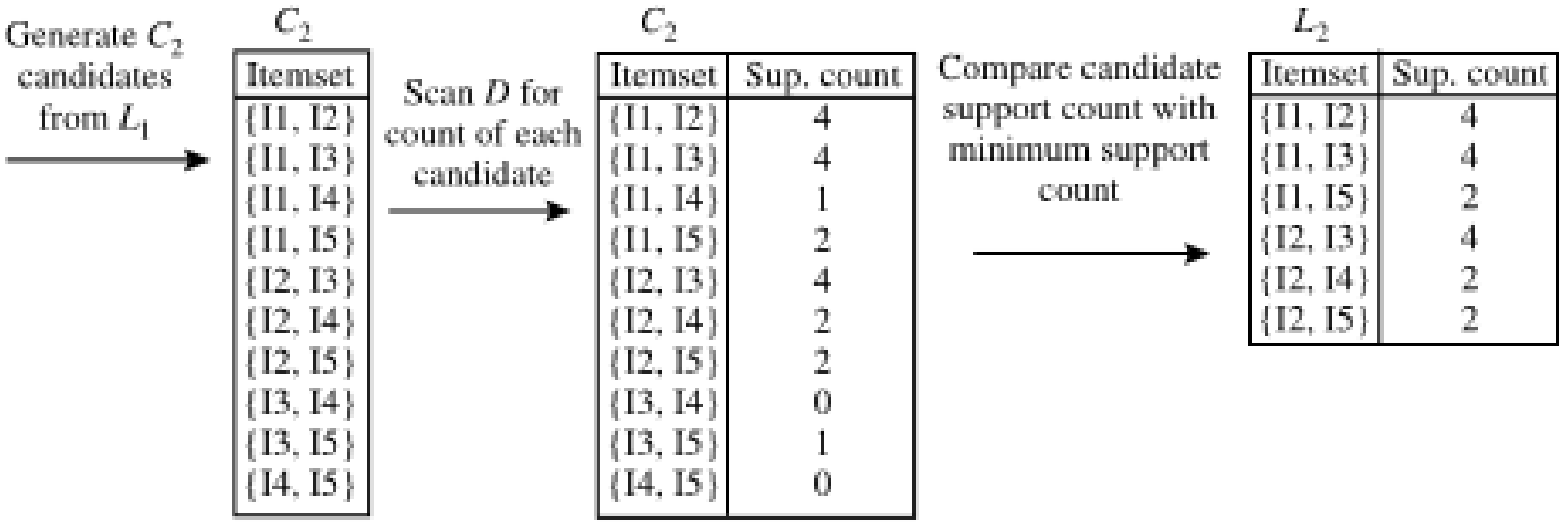
I1, I2,I3

I1,I2,I5

I1,I3,I5

I2,I3,I5

I2,I4,I5


Generate C_2 candidates from L_1
C_2
Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}
Scan D for count of each candidate
C_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I4} 1
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2
{I3, I4} 0
{I3, I5} 1
{I4, I5} 0
Compare candidate support count with minimum support count
L_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2

I1, I2,I3

I1,I2,I5

I1,I3,I5

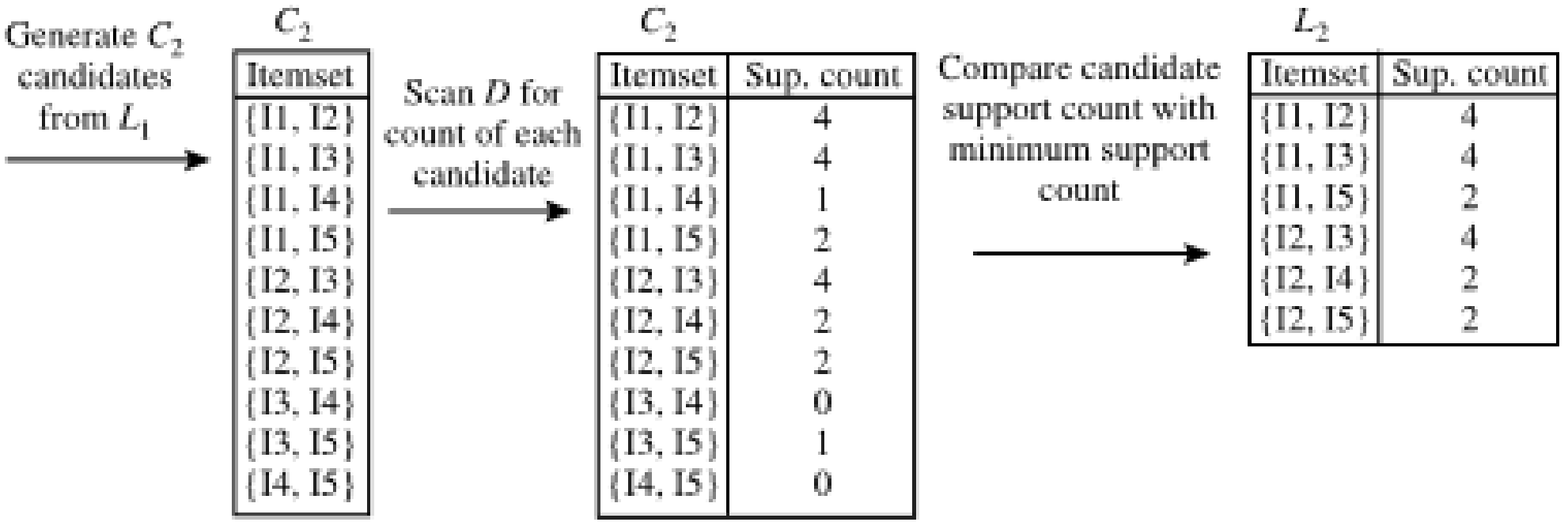
I2,I3,I5

I2,I4,I5

I1, I2,I3

I1,I2,I5

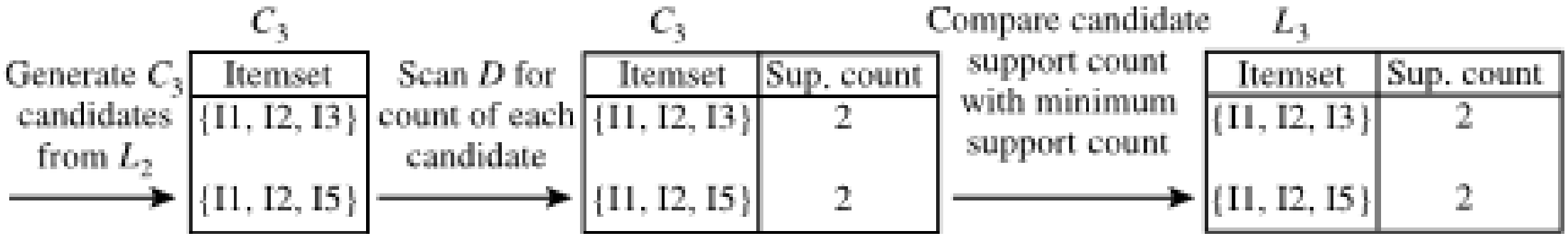
I2,I4,I5


Generate C_2 candidates from L_1
C_2
Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}
Scan D for count of each candidate
C_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I4} 1
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2
{I3, I4} 0
{I3, I5} 1
{I4, I5} 0
Compare candidate support count with minimum support count
L_2
Itemset
Sup. count
{I1, I2} 4
{I1, I3} 4
{I1, I5} 2
{I2, I3} 4
{I2, I4} 2
{I2, I5} 2

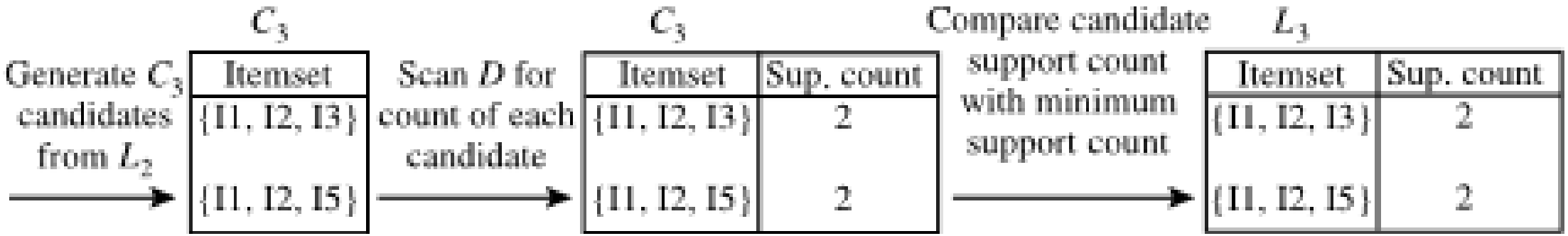
I1, I2,I3

I1,I2,I5

I2,I4,I5


C_3
Itemset
{I1, I2, I3}
{I1, I2, I5}
Generate C_3 candidates from L_2
Scan D for count of each candidate
C_3
Itemset
Sup. count
{I1, I2, I3}
2
{I1, I2, I5}
2
Compare candidate support count with minimum support count
L_3
Itemset
Sup. count
{I1, I2, I3}
2
{I1, I2, I5}
2

I1,I2,I3,I5


C_3
Itemset
{I1, I2, I3}
{I1, I2, I5}
Generate C_3 candidates from L_2
Scan D for count of each candidate
C_3
Itemset
Sup. count
{I1, I2, I3}
2
{I1, I2, I5}
2
Compare candidate support count with minimum support count
L_3
Itemset
Sup. count
{I1, I2, I3}
2
{I1, I2, I5}
2

Example of Apriori: Support threshold=50%, Confidence= 60%

TABLE-1

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

Solution:

Transaction

Support threshold=50% => 0.5*6= 3 => min_sup=3

T1

1. Count Of Each Item

TABLE-2

Item	Count
I1	4
I2	5
I3	4
I4	4
I5	2

2. Prune Step: TABLE -2 shows that I5 item does not meet min_sup=3, thus it is deleted, only I1, I2, I3, I4 meet min_sup count.

TABLE-3

Item	Count
I1	4
I2	5
I3	4
I4	4

3. Join Step: Form 2-itemset. From **TABLE-1** find out the occurrences of 2-itemset.

TABLE-4

Item	Count
I1,I2	4
I1,I3	3
I1,I4	2
I2,I3	4
I2,I4	3
I3,I4	2

4. Prune Step: TABLE -4 shows that item set {I1, I4} and {I3, I4} does not meet min_sup, thus it is deleted.

TABLE-5

Item	Count
I1,I2	4
I1,I3	3
I2,I3	4
I2,I4	3

Item	count
I1, I2, I3	3
I1, I2, I4	
I2, I3, I4	

{I1, I2, I3} is frequent

Assume the support count is 2 and minimum confidence value is **60%.**

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

C1

Itemset	Support
{1}	3
{2}	3
{3}	4
{4}	1
{5}	4

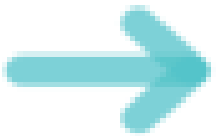
F1

Itemset	Support
{1}	3
{2}	3
{3}	4
{5}	4

Only Items present in F1

C2

Itemset	Support
{1,2}	1
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3



F2

Itemset	Support
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3

C_3

Itemset	In F2?
{1,2,3}, {1,2}, {1,3}, {2,3}	NO
{1,2,5}, {1,2}, {1,5}, {2,5}	NO
{1,3,5},{1,5}, {1,3}, {3,5}	YES
{2,3,5}, {2,3}, {2,5}, {3,5}	YES

F3

Itemset	Support
{1,3,5}	2
{2,3,5}	2

Frequent Itemset Generation, whose objective is to find frequent itemsets.

frequent itemset is an itemsets whose support is greater than or equal to minsup threshold

Rule Generation, whose objective is to extract rules that satisfy both minsup and minconf

These rules are called strong rules

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

Rule Generation

For l = {1,3,5}, subsets are {1,3}, {1,5}, {3,5}, {1}, {3}, {5}
For l = {2,3,5}, subsets are {2,3}, {2,5}, {3,5}, {2}, {3}, {5}

{1,3,5}

1,3→ 5	confidence= 2/3=66.6%	(selected)
1,5→3	confidence=2/2=100%	(selected)
3,5→1	confidence=2/3=66.6%	(selected)
1→3,5	confidence=2/3=66.6%	(selected)
3→1,5	confidence=2/4=50%	(Rejected)
5→1,3	confidence=2/4=50%	(Rejected)

{2,3,5}

2,3→ 5	confidence= 2/2=100%	(selected)
2,5→3	confidence=2/3=66.6%	(selected)
3,5→2	confidence=2/3=66.6%	(selected)
2→3,5	confidence=2/3=66.6%	(selected)
3→2,5	confidence=2/4=50%	(Rejected)
5→2,3	confidence=2/4=50%	(Rejected)

Candidate Generation

→There are many ways to generate candidate itemsets.

→candidate generation procedure must be complete and non-redundant.

Several candidate generation procedure:

Brute force method

$F_{k-1} \times F_1$ Method

$F_{k-1} \times F_{k-1}$ Method

Brute-Force Method

The brute-force method considers every k-itemset as a potential candidate and then applies the candidate pruning step to remove any unnecessary candidates whose subsets are infrequent

→Extremely Expensive

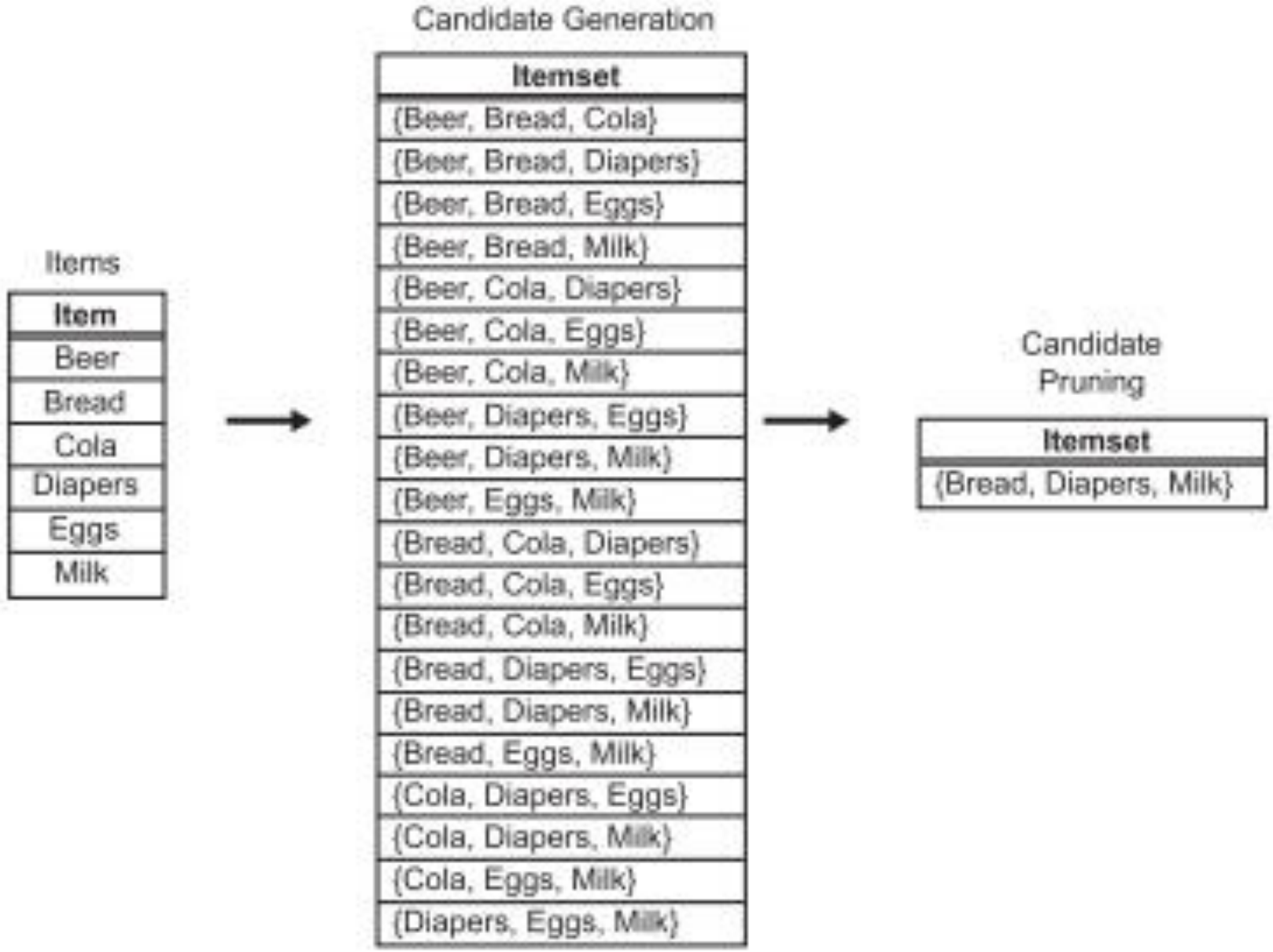
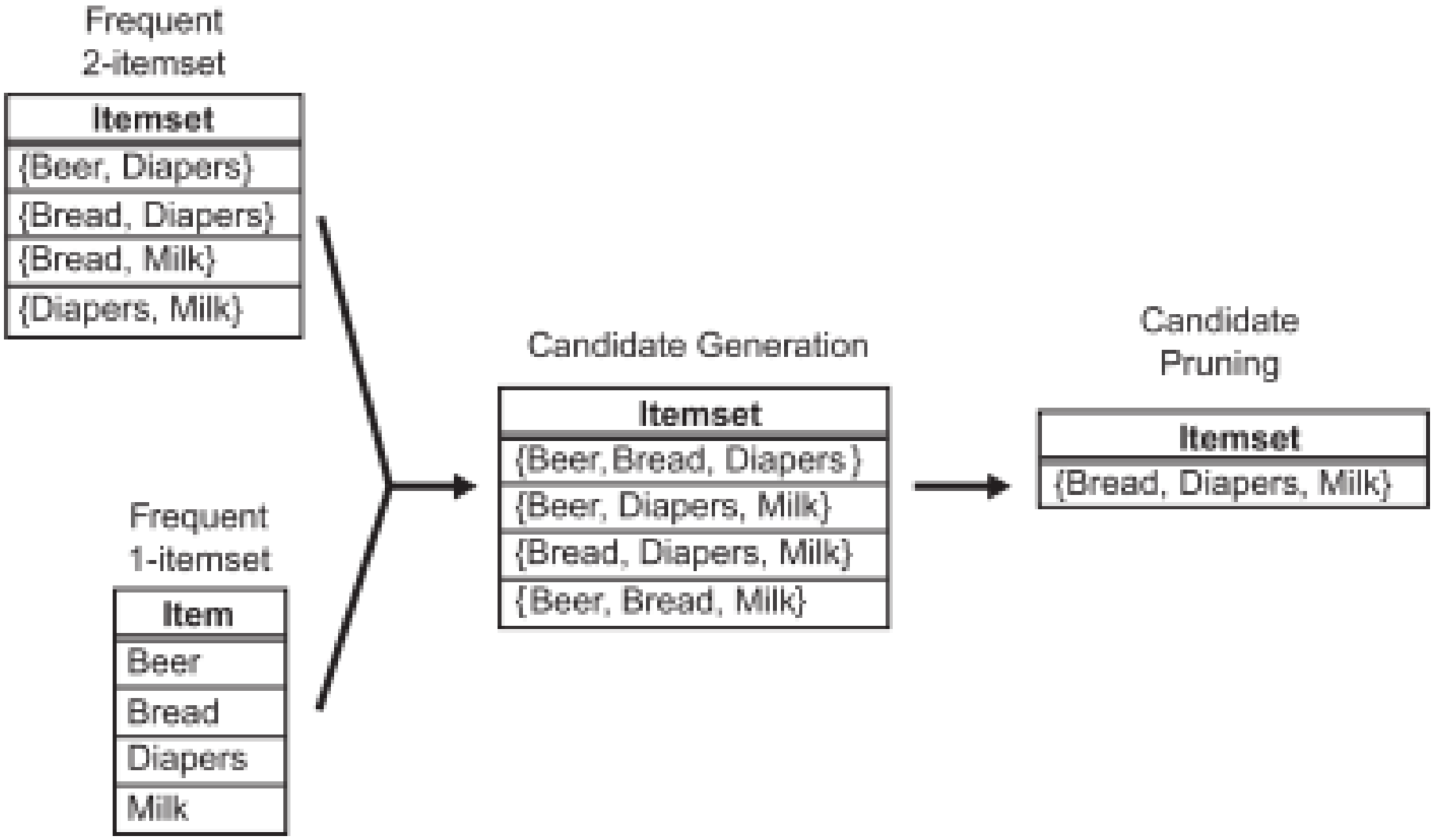


Figure 5.6. A brute-force method for generating candidate 3-itemsets.

$F_{k-1} \times F_1$ Method

→An alternative method for candidate generation



re 5.7. Generating and pruning candidate k-itemsets by merging a frequent $(k-1)$-itemset with a ient item. Note that some of the candidates are unnecessary because their subsets are infrequent.

Drawback:

Does not prevent the same candidate itemset from being generated more than once.

For instance,

{Bread, Diapers, Milk} can be generated by merging {Bread, Diapers} with {Milk}, {Bread, Milk} with {Diapers}, or {Diapers, Milk} with {Bread}.

Overcome:

Frequent itemset are kept sorted in their lexicographic order.

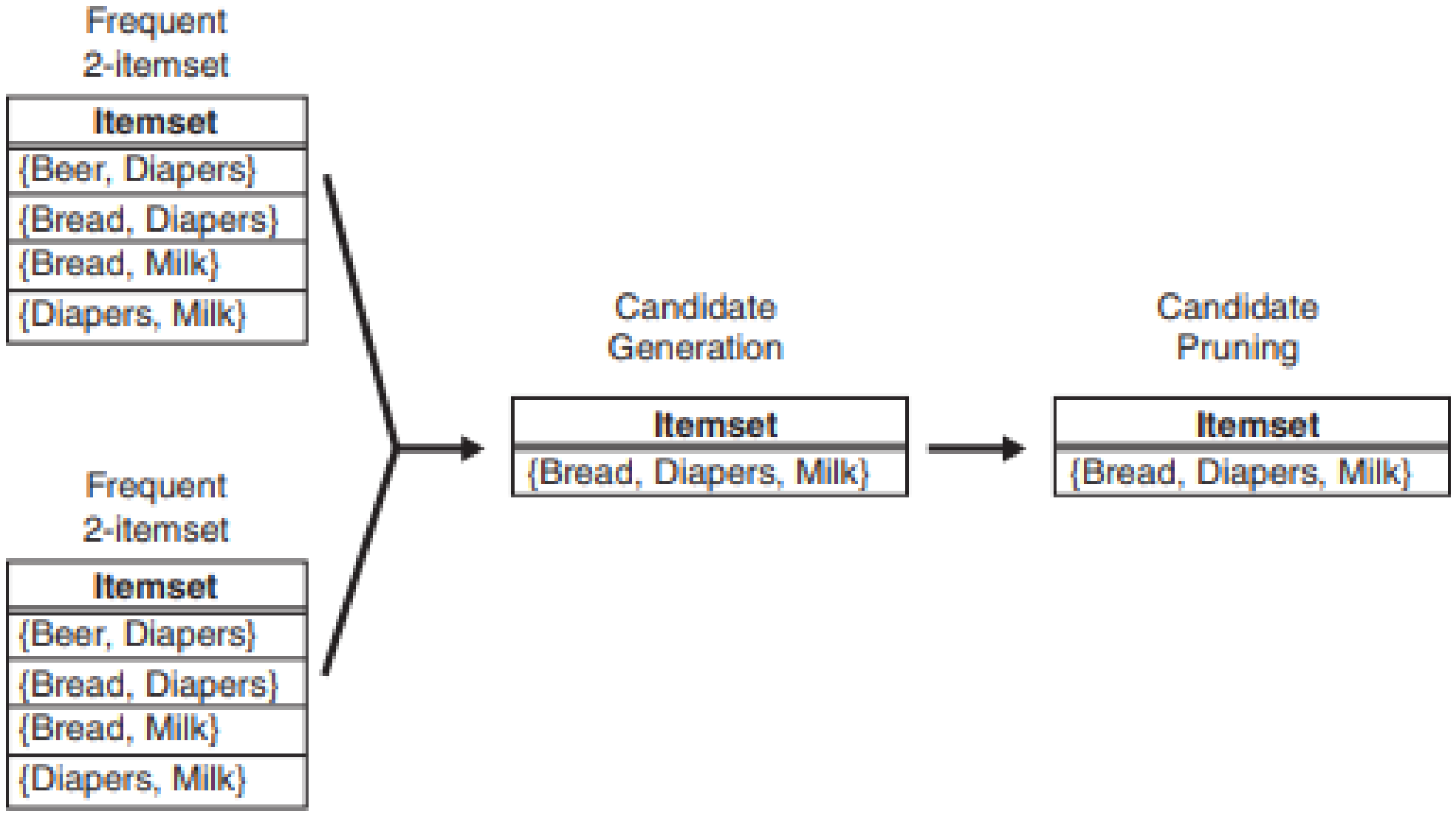


Figure 5.8. Generating and pruning candidate k-itemsets by merging pairs of frequent $(k-1)$-itemsets.

Pseudo code for Frequent Itemset generation of the Apriori Algorithm:

Method:

- Let k=1
- Generate frequent itemsets of length 1
- Repeat until no new frequent itemsets are identified
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Prune candidate itemsets containing subsets of length k that are infrequent
  - Count the support of each candidate by scanning the DB
  - Eliminate candidates that are infrequent, leaving only those that are frequent

```
k=1
F[1] = all frequent 1-itemsets
while F[k] != Empty Set:
    k += 1
    ## Candidate Itemsets Generation and Pruning
    C[k] = candidate itemsets generated by F[k-1]

    ## Support Counting
    for transaction t in T:
        C[t] = subset(C[k], t)     # Identify all candidates that belong to t
        for candidate itemset c in C[t]:
            support_count[c] += 1  # Increment support count

    F[k] = {c | c in C[k] and support_count[c]>=N*minsup}

return F[k] for all values of k
```

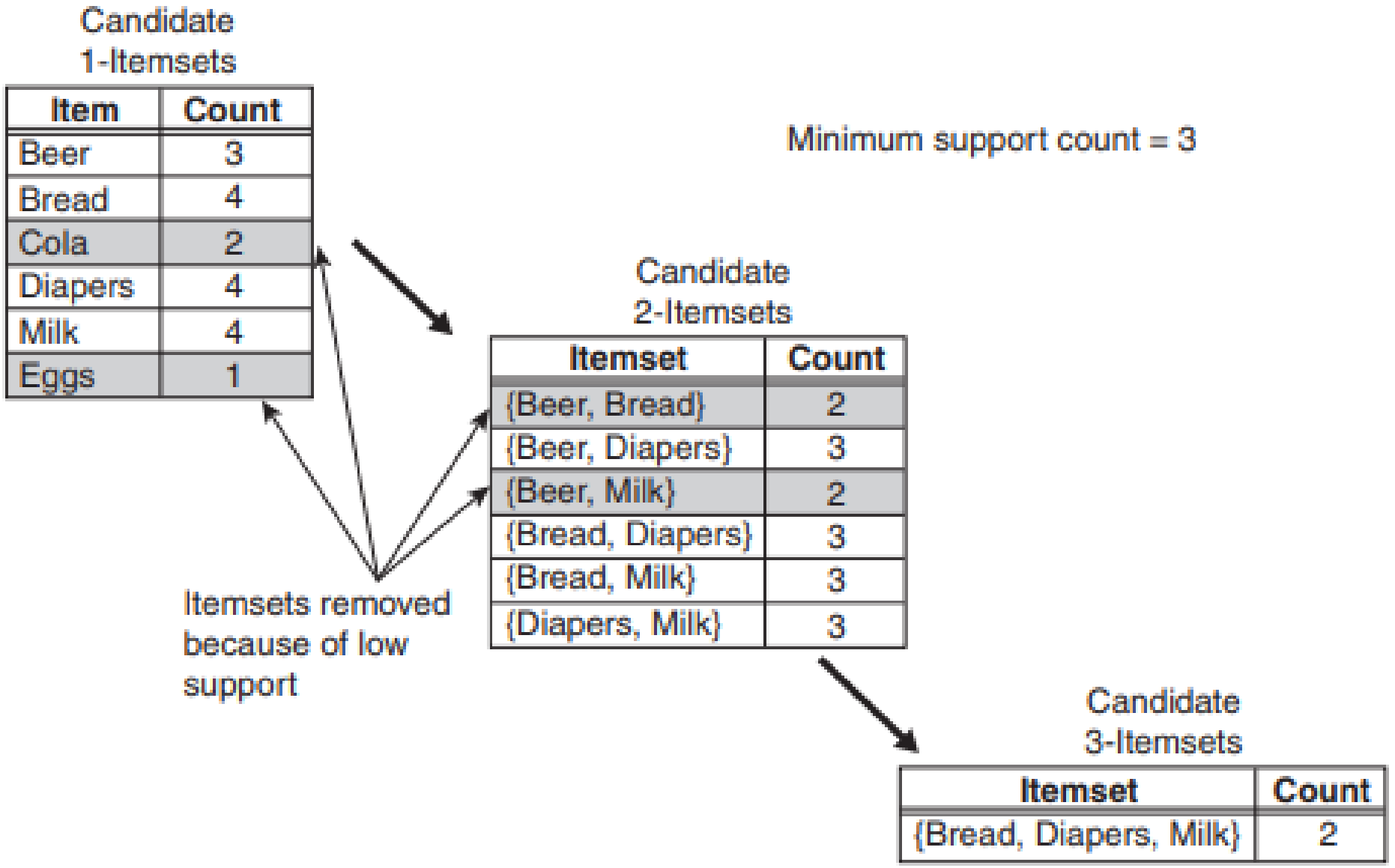


Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

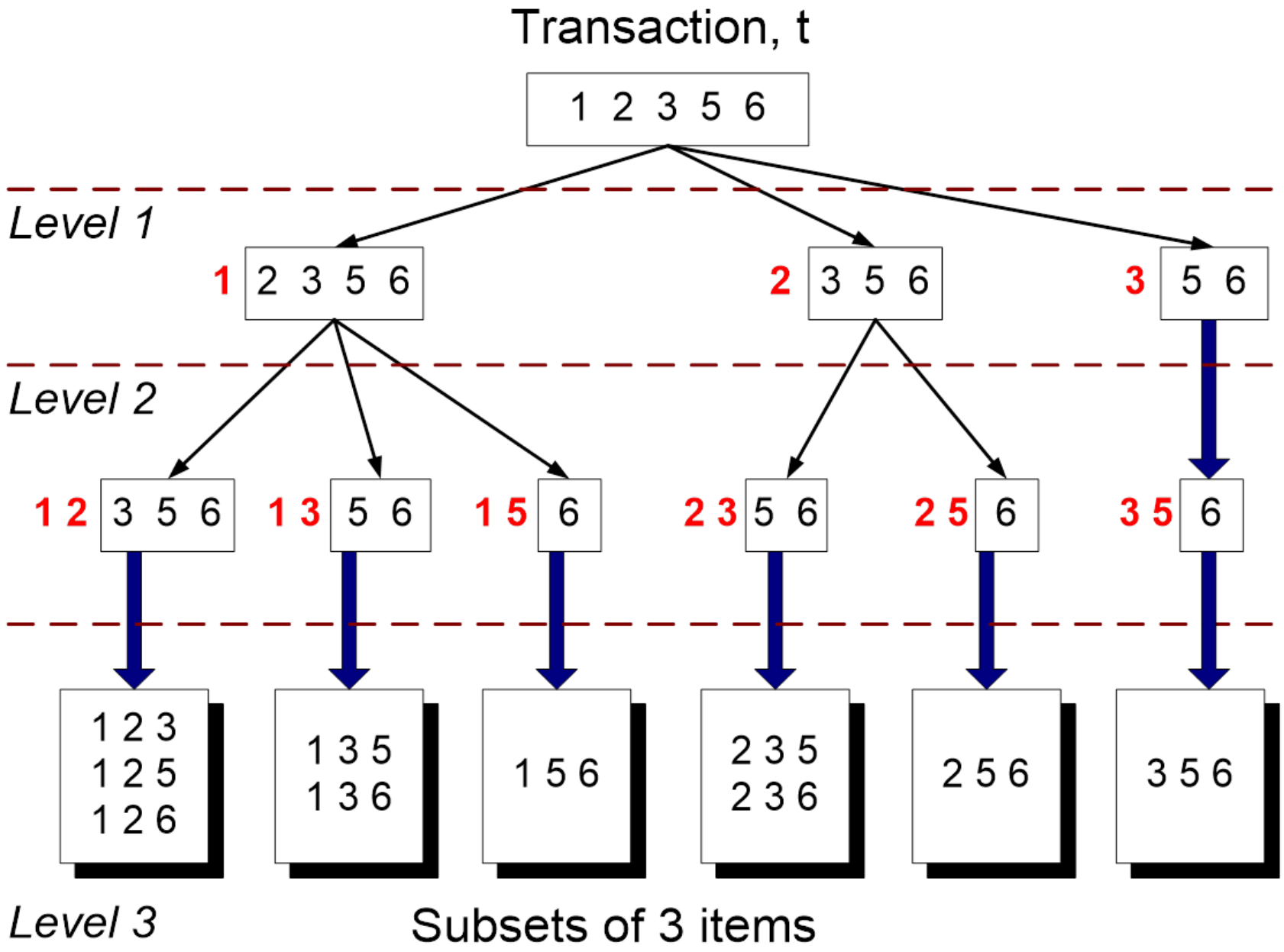
Support counting

Support counting is the process of determining the frequency of occurrence for every candidate itemset that survives the candidate pruning step

One approach for doing this is to compare each transaction against every candidate itemset

→ Computationally expensive

Alternate approach is to enumerate the itemsets contained in each transaction and use them to update the support counts


Transaction, t
1 2 3 5 6
Level 1
1 2 3 5 6
2 3 5 6
3 5 6
Level 2
1 2 3 5 6
1 3 5 6
1 5 6
2 3 5 6
2 5 6
3 5 6
1 2 3
1 2 5
1 2 6
1 3 5
1 3 6
1 5 6
2 3 5
2 3 6
2 5 6
3 5 6
Level 3
Subsets of 3 items

Support count using a hash tree

Candidate itemsets are partitioned into different buckets and stored in a hash tree.

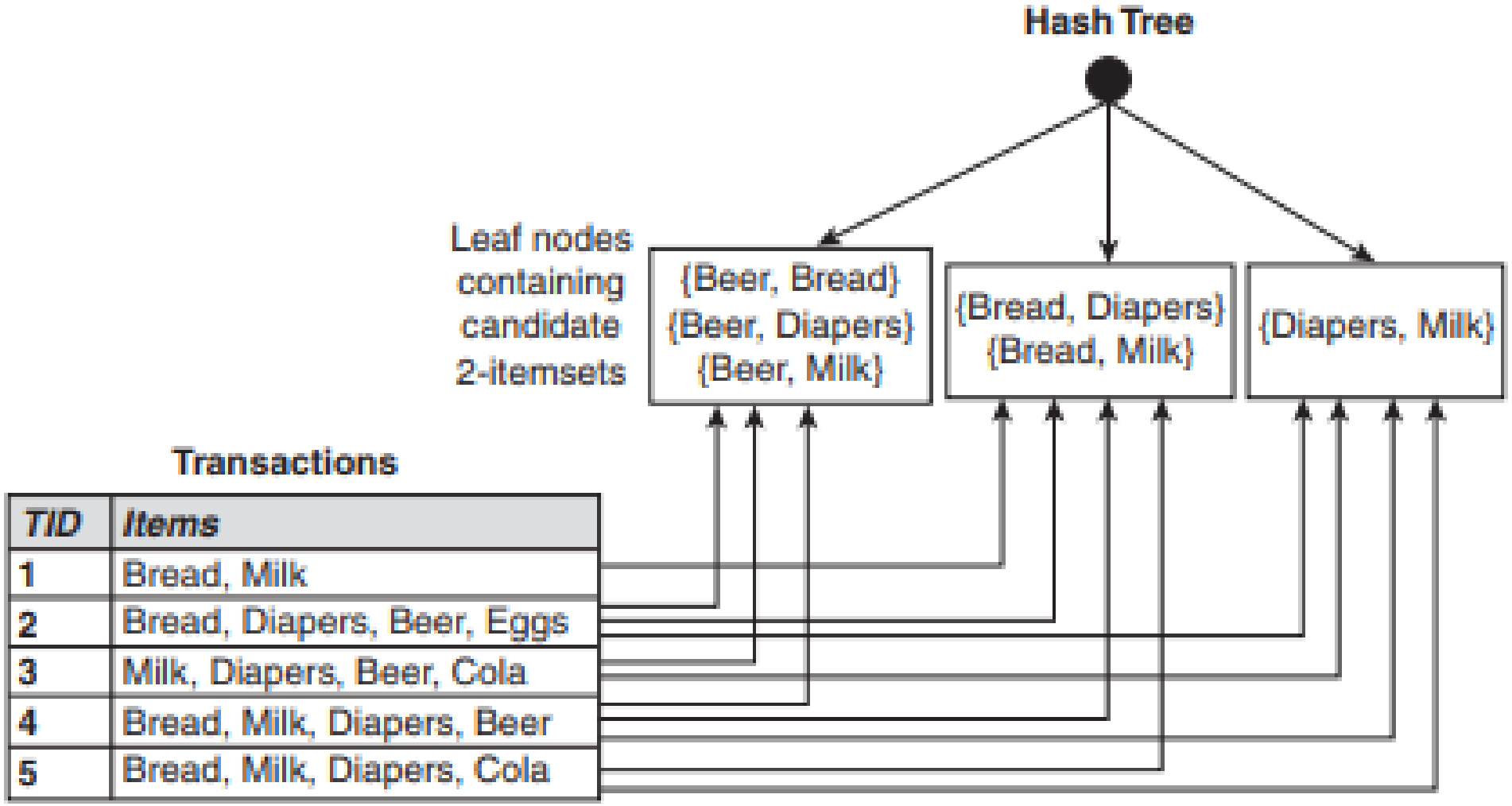


Figure 5.10. Counting the support of itemsets using hash structure.

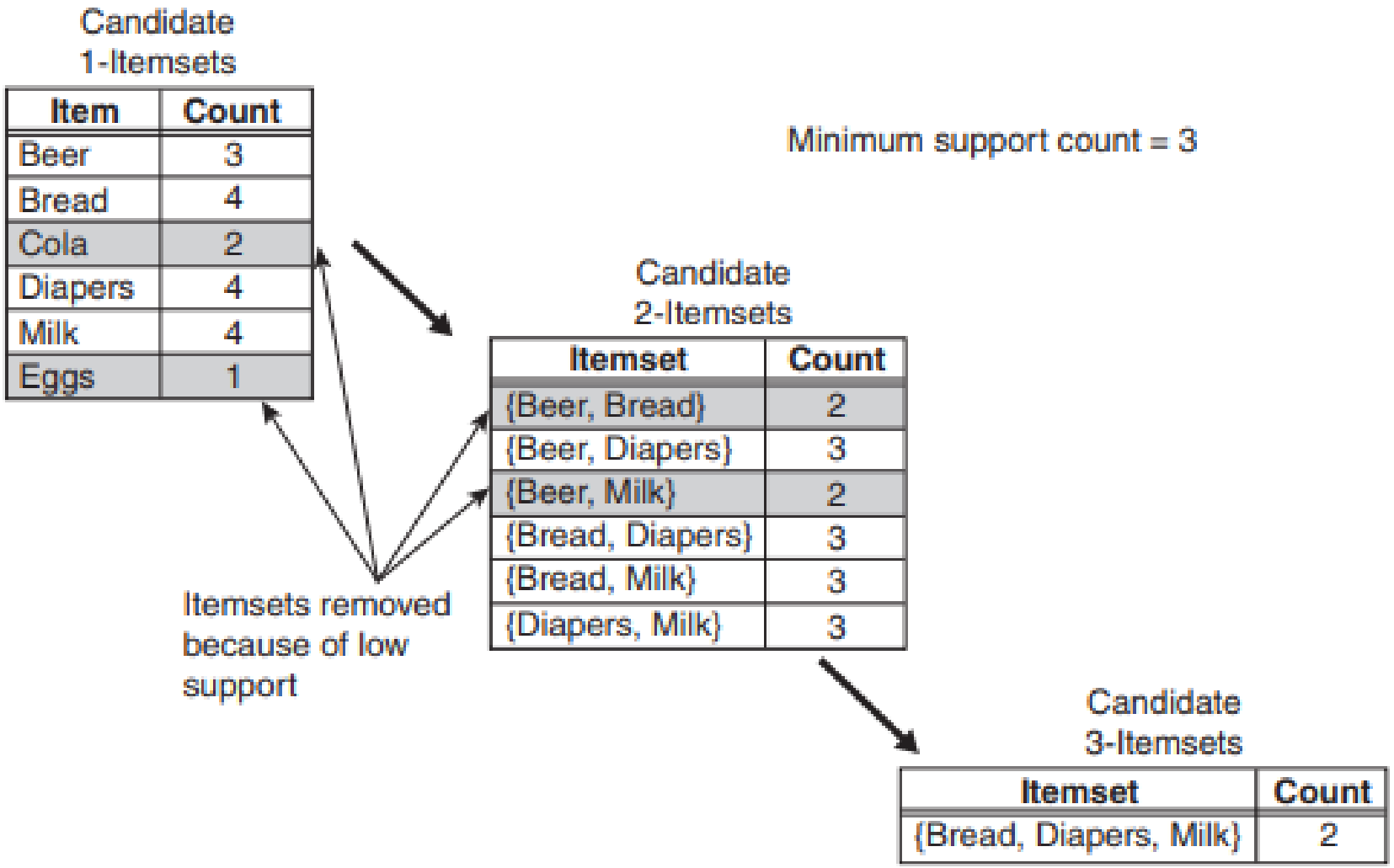


Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Support counting using hash tree

In order to generate hash tree structure, we need a hash function

$h(p) = p \bmod 3$

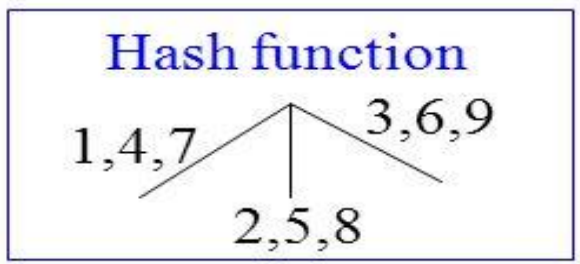
Generate Hash Tree

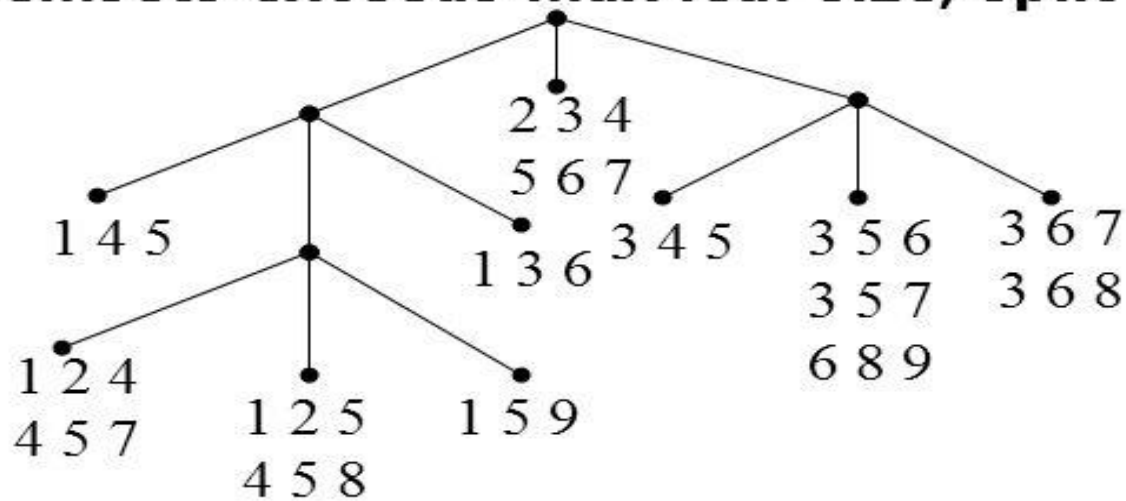
Suppose you have 15 candidate itemsets of length 3:

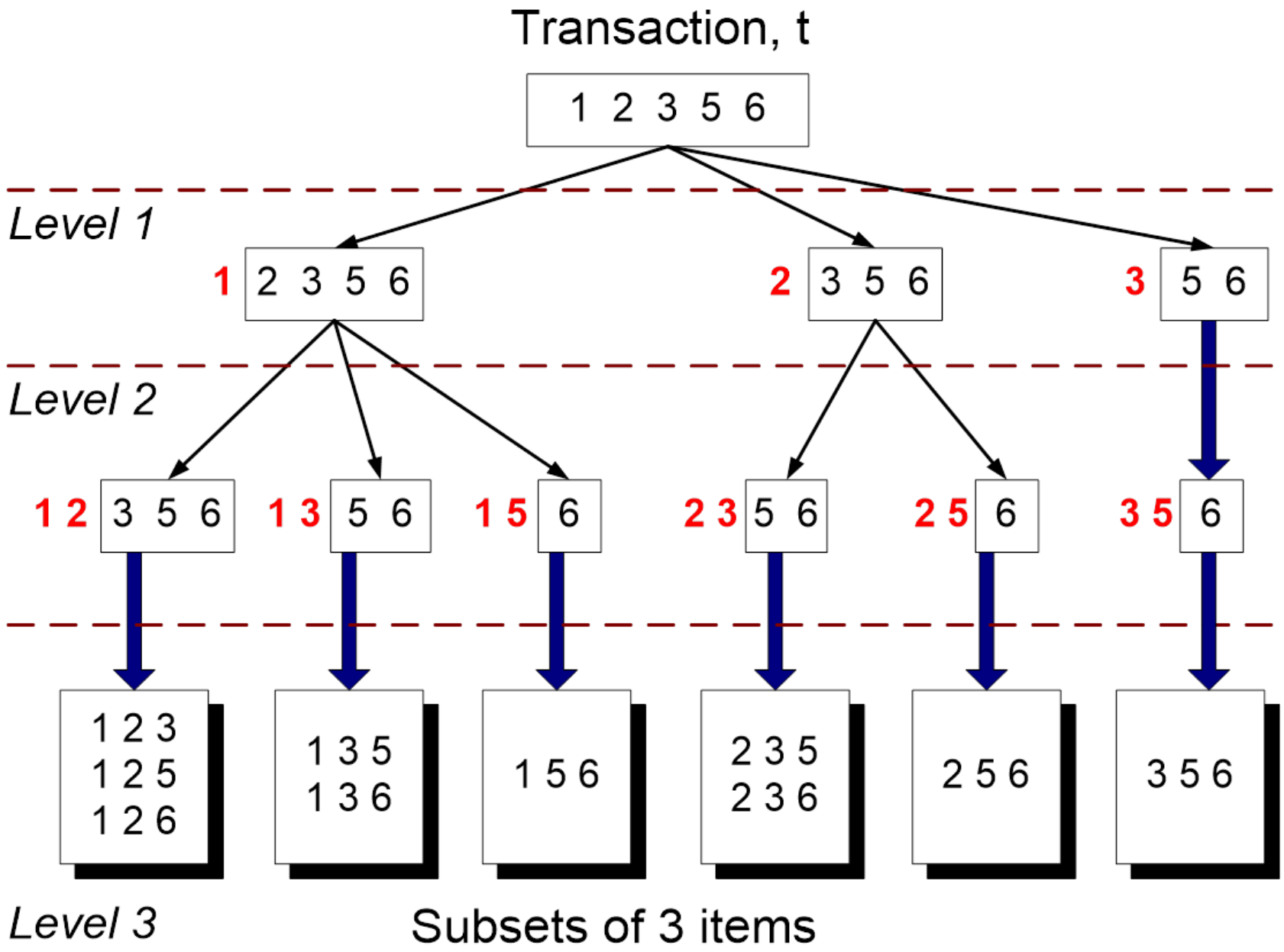
{1 4 5}, {1 2 4}, {4 5 7}, {1 2 5}, {4 5 8}, {1 5 9}, {1 3 6}, {2 3 4}, {5 6 7}, {3 4 5}, {3 5 6}, {3 5 7}, {6 8 9}, {3 6 7}, {3 6 8}

You need:

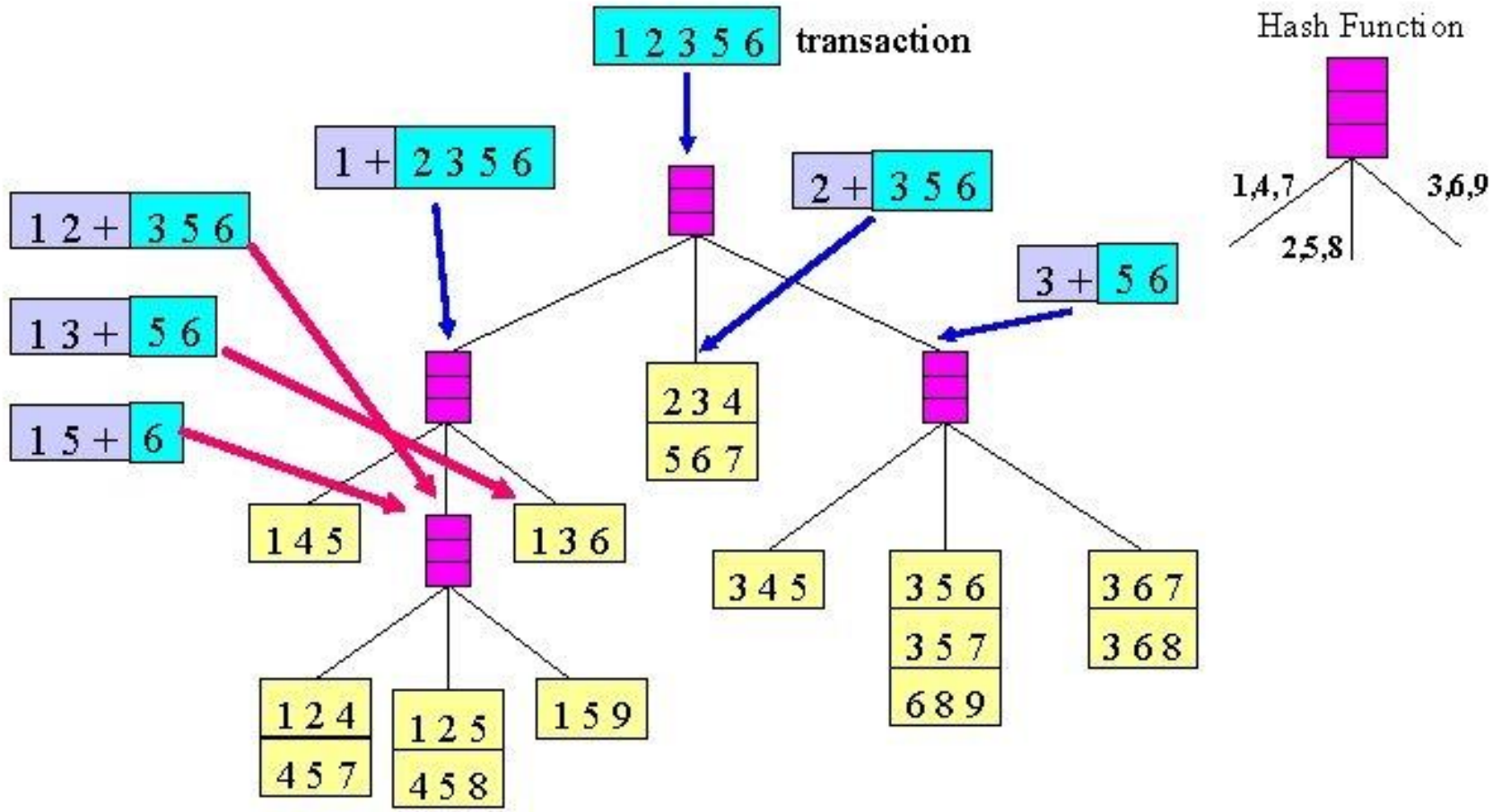
- **Hash function**
- **Max leaf size: max number of itemsets stored in a leaf node (if number of candidate itemsets exceeds max leaf size, split the node)**






Transaction, t
1 2 3 5 6
Level 1
1 2 3 5 6
2 3 5 6
3 5 6
Level 2
1 2 3 5 6
1 3 5 6
1 5 6
2 3 5 6
2 5 6
3 5 6
1 2 3
1 2 5
1 2 6
1 3 5
1 3 6
1 5 6
2 3 5
2 3 6
2 5 6
3 5 6
Level 3
Subsets of 3 items

Support Counting Using a Hash Tree



```
k=1
F[1] = all frequent 1-itemsets
while F[k] != Empty Set:
    k += 1
    ## Candidate Itemsets Generation and Pruning
    C[k] = candidate itemsets generated by F[k-1]

    ## Support Counting
    for transaction t in T:
        C[t] = subset(C[k], t)     # Identify all candidates that belong to t
        for candidate itemset c in C[t]:
            support_count[c] += 1  # Increment support count

    F[k] = {c | c in C[k] and support_count[c]>=N*minsup}

return F[k] for all values of k
```

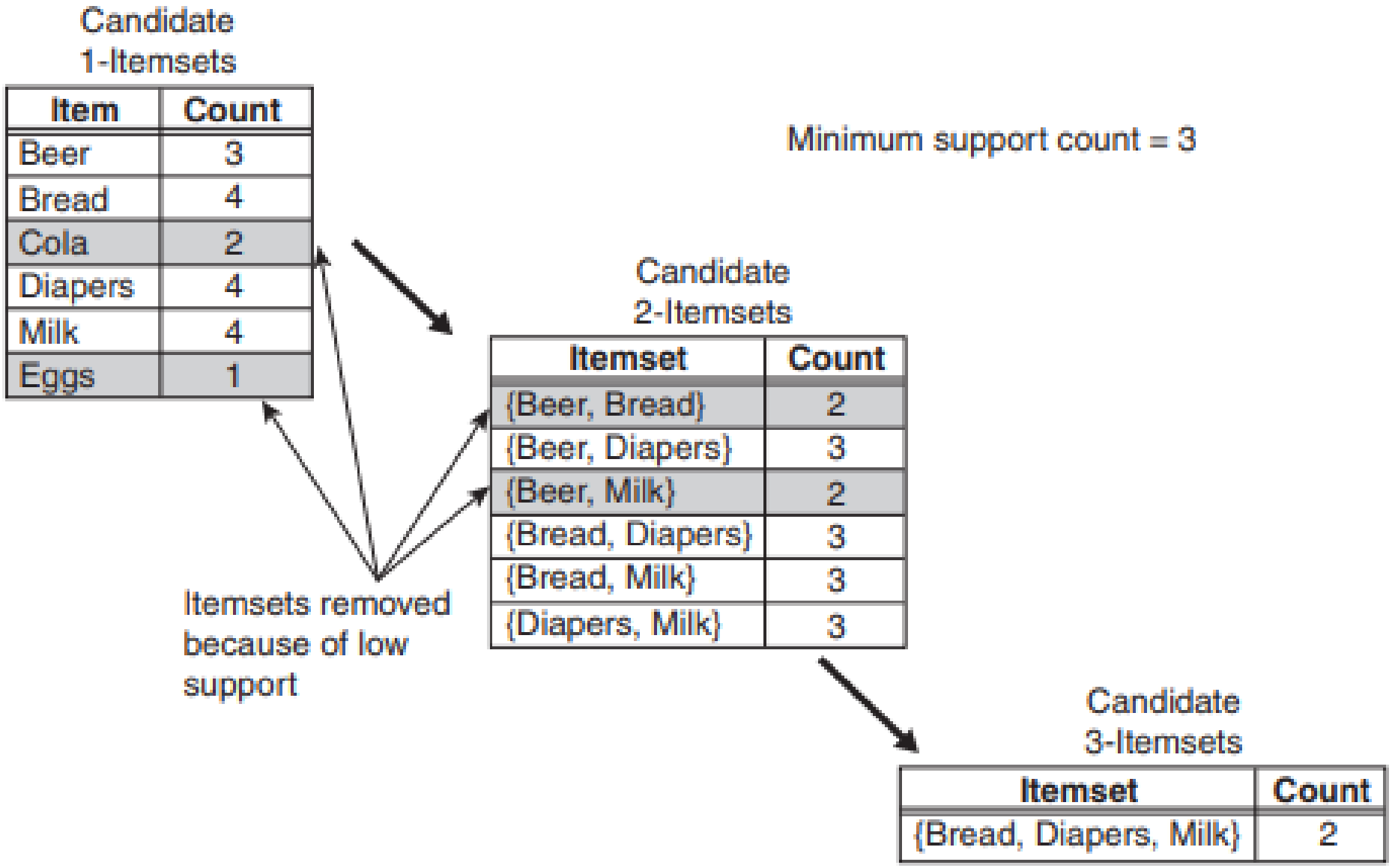


Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

Computational complexity

The computational complexity of the Apriori Algorithm can be affected by following factors:

Support Threshold

Number of Items(Dimensionality)

Number of Transactions

Average Transaction width

Generation of frequent 1-Itemset

Support Counting

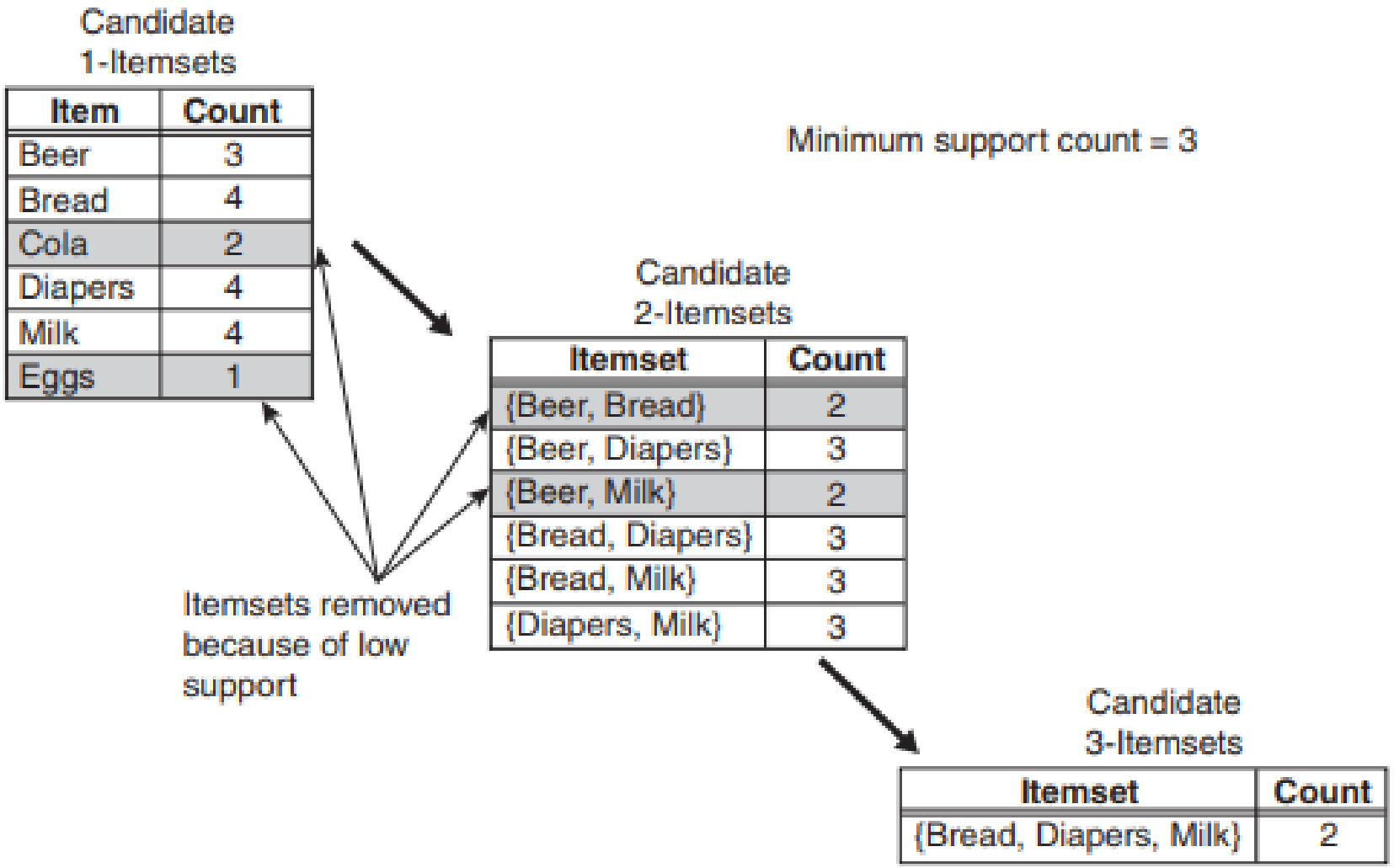


Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

TID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}

Support Threshold

Lowering the support threshold often results in more itemsets being declared as frequent.

This increases the total number of iterations to be performed by the Apriori algorithm, further increasing the computational cost.

Number of Items (Dimensionality)

As the number of items increases, more space will be needed to store the support counts of items.

The number of frequent items also grows with the dimensionality of the data

The runtime and storage requirements will increase because of the larger number of candidate itemsets generated by the algorithm.

Number of Transactions

Because the Apriori algorithm makes repeated passes over the transaction data set, its run time increases with a larger number of transactions.

Average Transaction Width

For dense data sets, the average transaction width can be very large.

The maximum size of frequent itemsets tends to increase as the average transaction width increases.

Generation of frequent 1-itemsets

For each transaction, we need to update the support count for every item present in the transaction.

Assuming that w is the average transaction width, this operation requires O(Nw) time, where N is the total number of transactions.

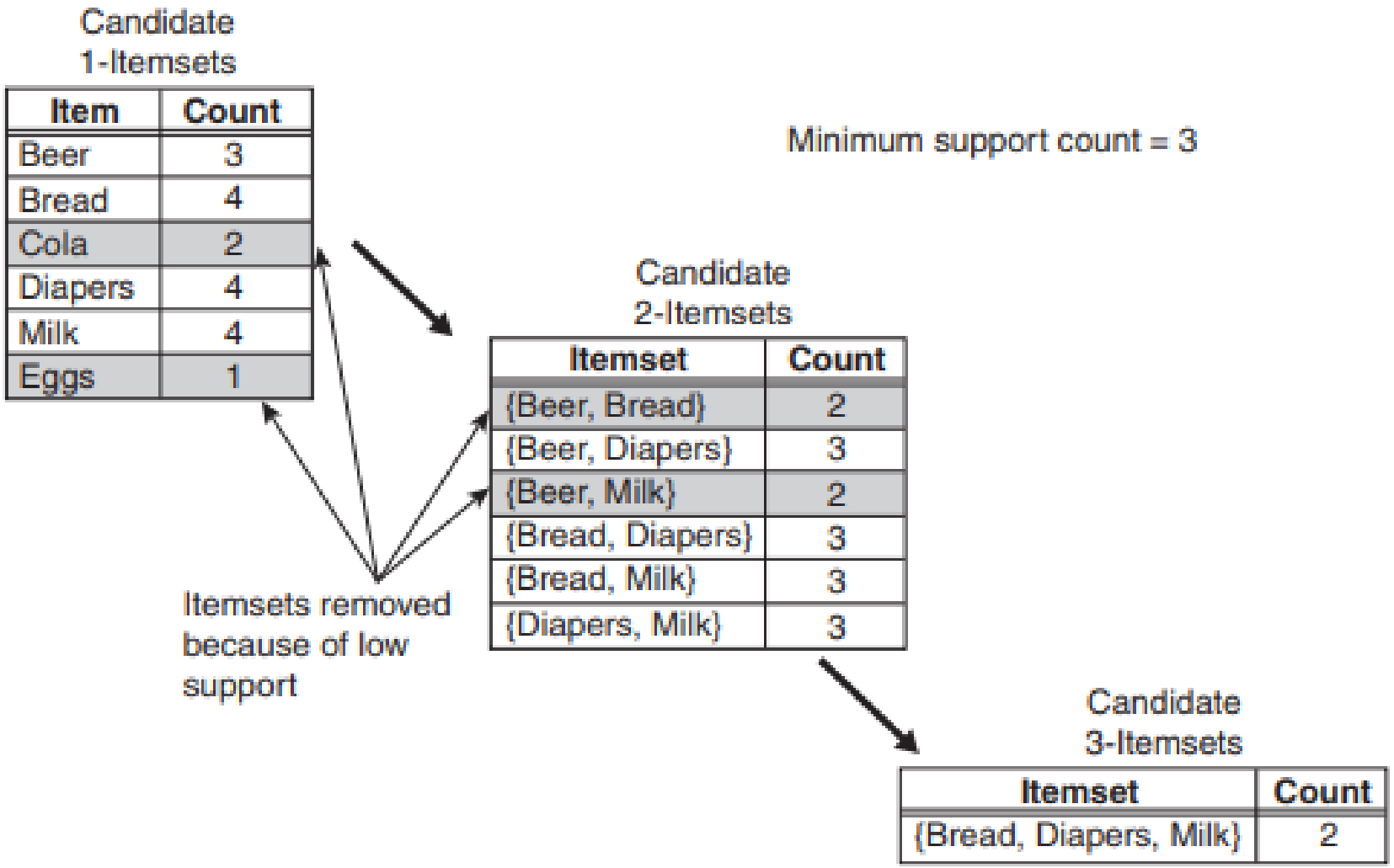


Figure 5.5. Illustration of frequent itemset generation using the *Apriori* algorithm.

Support counting

The cost for support counting is

$$O\left(N \sum_k \binom{w}{k} \alpha_k\right)$$

where w is the maximum transaction width
αk is the cost for updating the support count of a candidate k-itemset in the hash tree.

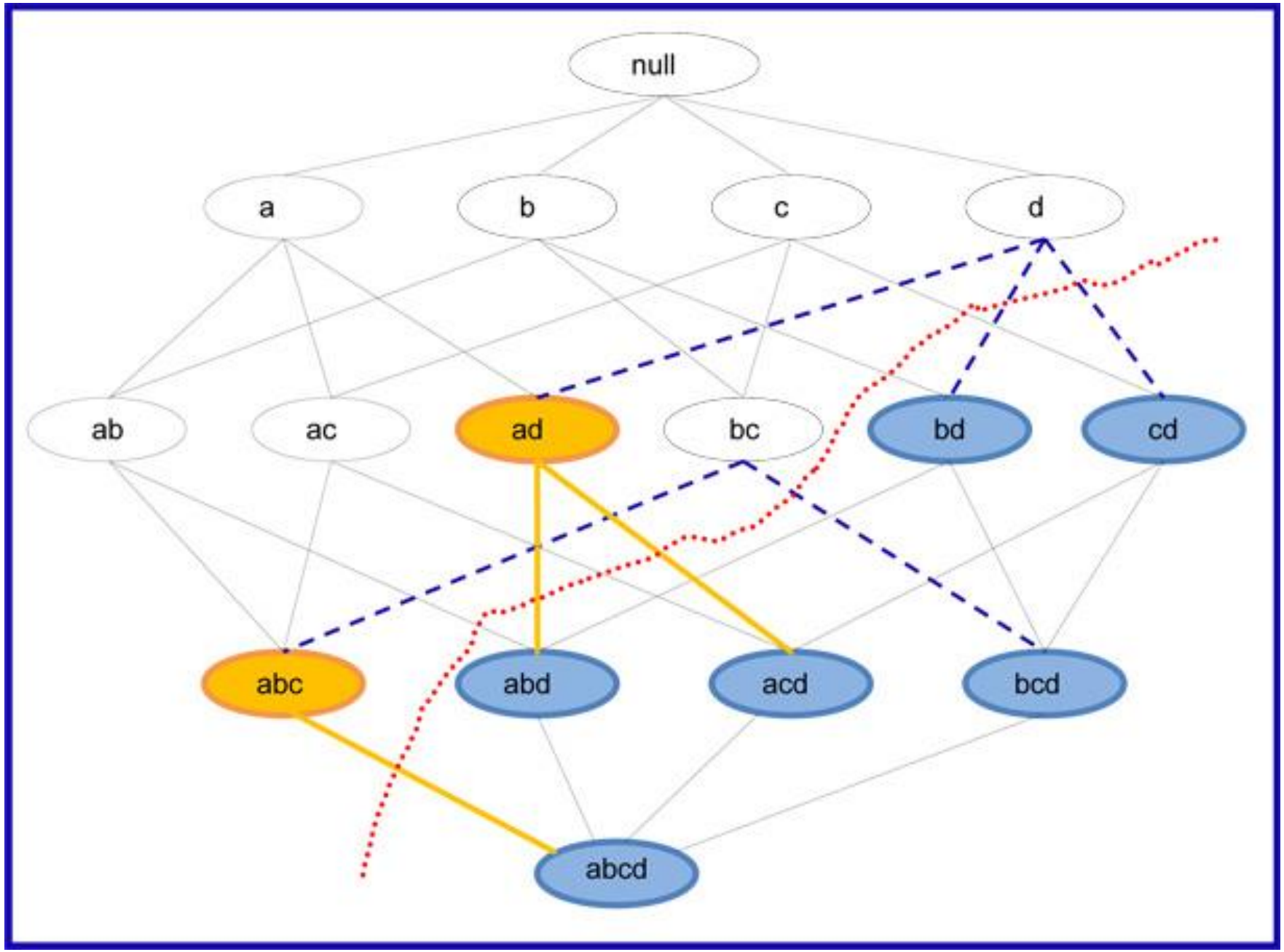
Compact Representation of Frequent Itemset

Maximal Frequent Itemset

Closed Frequent Itemset

Maximal Frequent Itemset

A frequent itemset is maximal if none of its immediate supersets are frequent.


null
a
b
c
d
ab
ac
ad
bc
bd
cd
abc
abd
acd
bcd
abcd

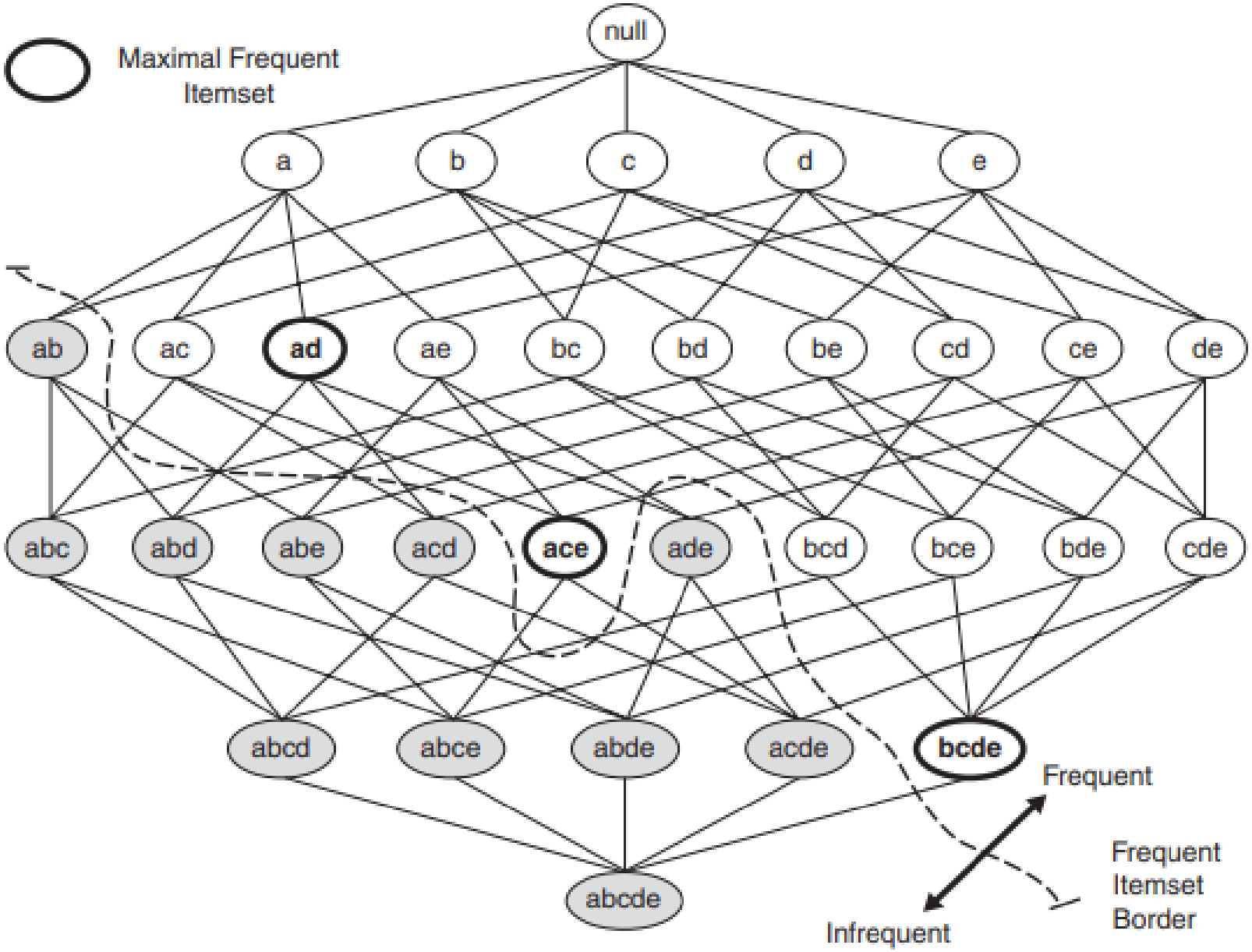


Figure 5.16. Maximal frequent itemset.

Closed Itemset

An itemset X is closed if none of its immediate supersets has exactly the same support count as X.

Closed Frequent Itemset

An itemset is a closed frequent itemset if it is closed and its support is greater than or equal to minsup.

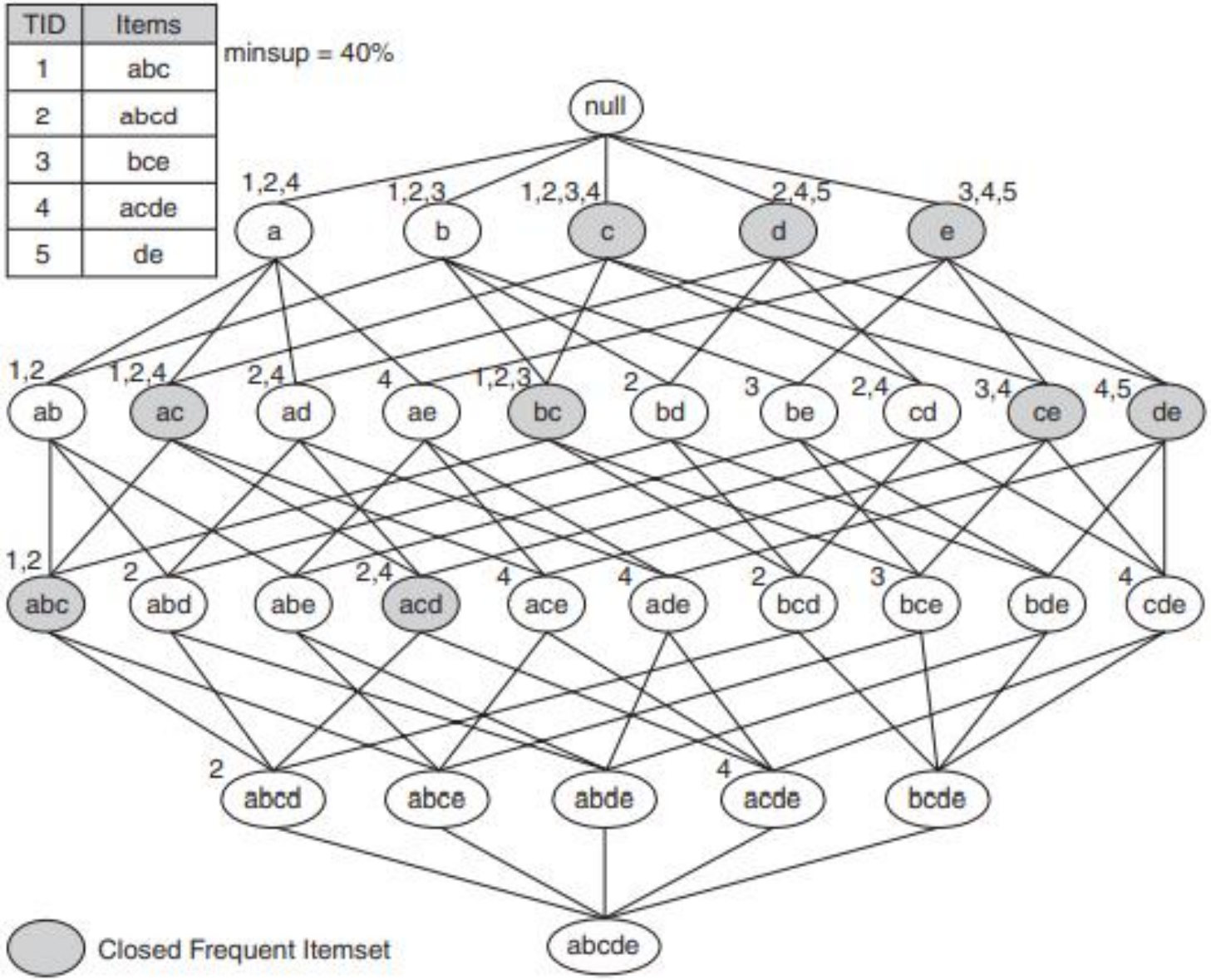


Figure 5.17. An example of the closed frequent itemsets (with minimum support equal to 40%).

Rule Generation

Confidence-Based Pruning

Theorem

Let Y be an itemset and X is a subset of Y .

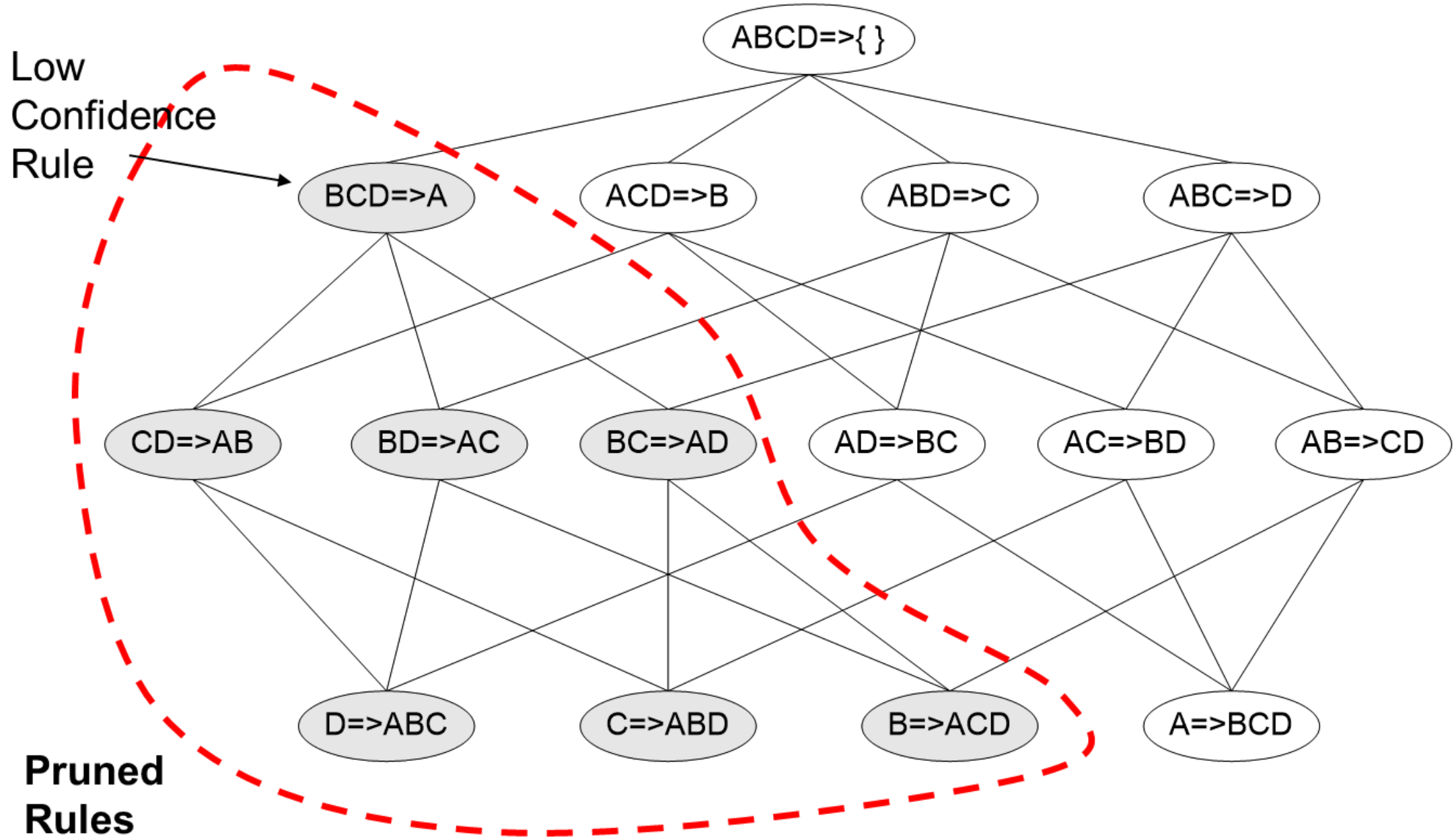
If a rule X → Y − X does not satisfy the confidence threshold, then any rule X’ → Y − X’, where X’ is a subset of X, must not satisfy the confidence threshold as well.

Rule Generation

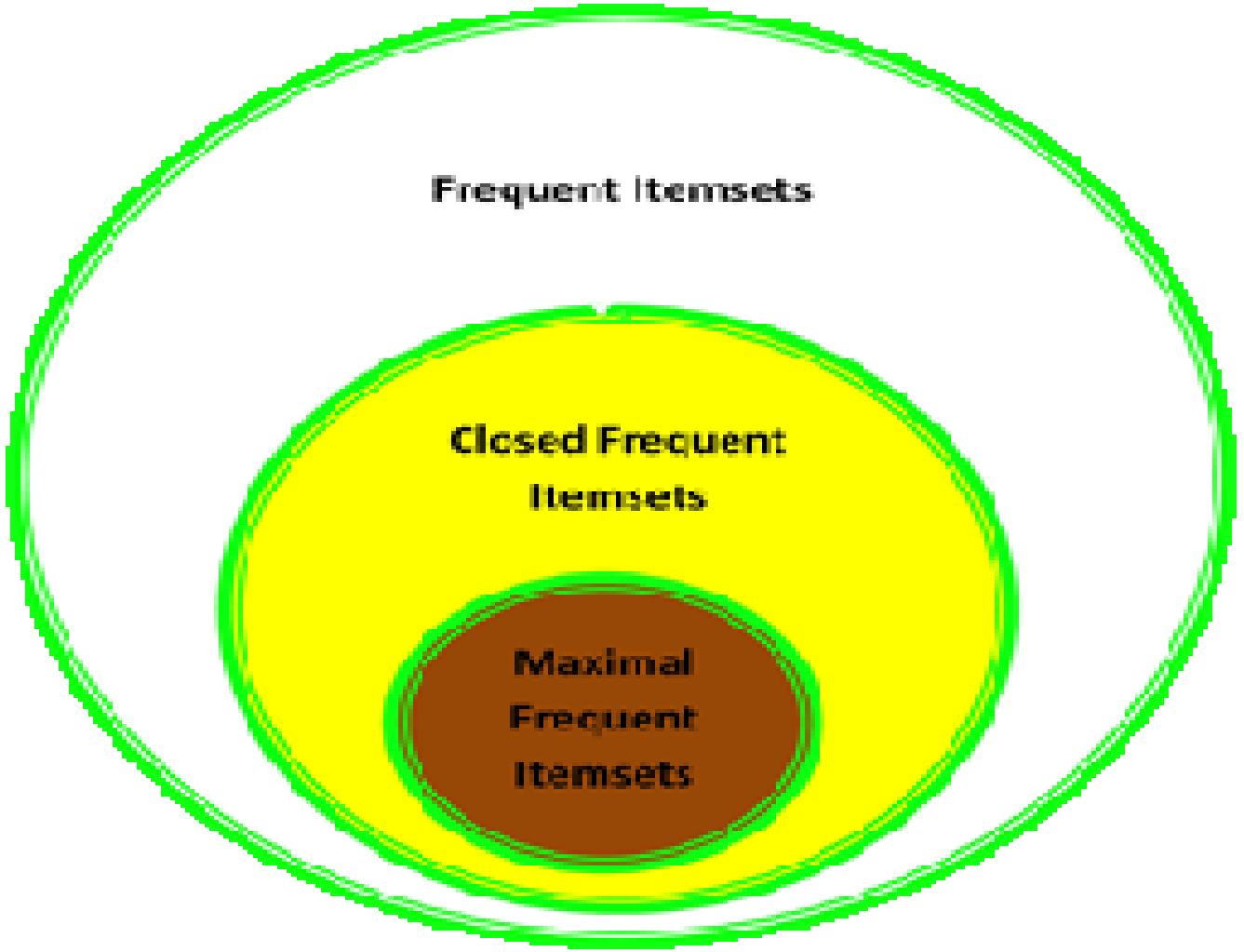
For l = {1,3,5}, subsets are {1,3}, {1,5}, {3,5}, {1}, {3}, {5}
For l = {2,3,5}, subsets are {2,3}, {2,5}, {3,5}, {2}, {3}, {5}

{1,3,5}

1,3→ 5	confidence= 2/3=66.6%	(selected)
1,5→3	confidence=2/2=100%	(selected)
3,5→1	confidence=2/3=66.6%	(selected)
1→3,5	confidence=2/3=66.6%	(selected)
3→1,5	confidence=2/4=50%	(Rejected)
5→1,3	confidence=2/4=50%	(Rejected)


Low
Confidence
Rule
ABCD=>{ }
BCD=>A
ACD=>B
ABD=>C
ABC=>D
CD=>AB
BD=>AC
BC=>AD
AD=>BC
AC=>BD
AB=>CD
D=>ABC
C=>ABD
B=>ACD
A=>BCD
Pruned
Rules

Relationship among frequent, maximal frequent and closed frequent itemset



FP-Growth Algorithm

→Finding frequent Itemset without candidate generation

→First compress the database into frequent-pattern tree, or FP- Tree

→Then divide the set of compressed database into a set of conditional database

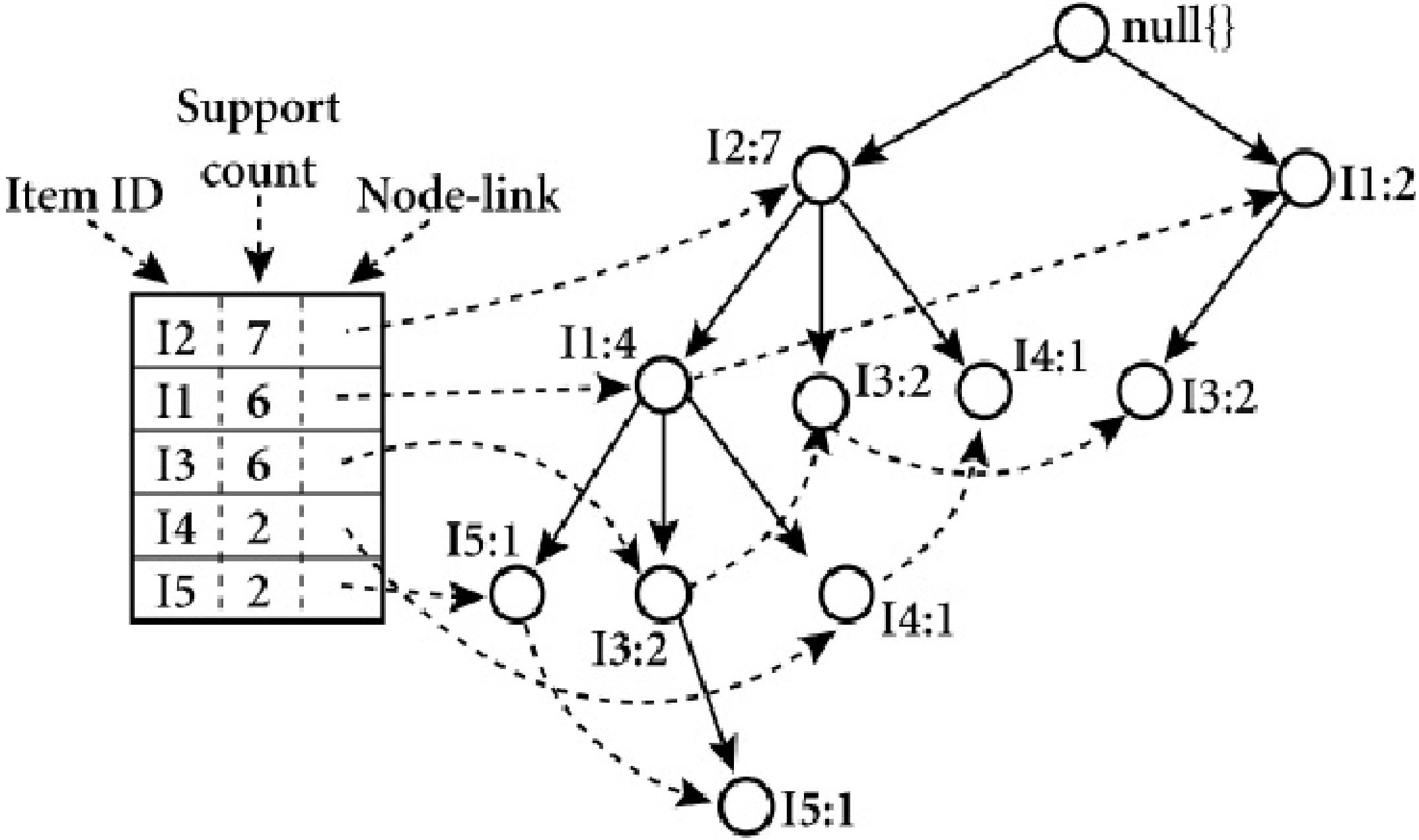
TID	List of Items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

minsup=2

Itemset	Support count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Itemset	Support count
{I2}	7
{I1}	6
{I3}	6
{I4}	2
{I5}	2

TID	List of Items	Ordered Items
T100	I1, I2, I5	I2, I1, I5
T200	I2, I4	I2, I4
T300	I2, I3	I2, I3
T400	I1, I2, I4	I2, I1, I4
T500	I1, I3	I1, I3
T600	I2, I3	I2, I3
T700	I1, I3	I1, I3
T800	I1, I2, I3, I5	I2, I1, I3, I5
T900	I1, I2, I3	I2, I1, I3


Item ID
Support count
Node-link
I2 7
I1 6
I3 6
I4 2
I5 2
null{}
I2:7
I1:2
I1:4
I3:2
I4:1
I3:2
I5:1
I3:2
I4:1
I5:1

Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}		
I4	{{I2, I1: 1}, {I2: 1}}		
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}		
I1	{I2: 4}		
I2	---	---	---

Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	<I2:2, I1:2>	
I4	{{I2, I1: 1}, {I2: 1}}	<I2:2>	
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	<I2:4, I1:2>, <I1:2>	
I1	{I2: 4}	<I2:4>	
I2	---	---	---

Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	<I2:2, I1:2>	{I2, I5: 2}, {I1,I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	<I2:2>	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	<I2:4, I1:2>, <I1:2>	{I2, I3: 4}, {I1,I3: 4}, {I1, I2, I3: 2}
I1	{I2: 4}	<I2:4>	{I2, I1: 4}
I2	---	---	---

TID	Itemset
100	{ M, O, N, K, E, Y }
200	{ D, O, N, K, E, Y }
300	{ M, A, K, E }
400	{ M, U, C, K, Y }
500	{ C, O, K, I, E }

minsup=3

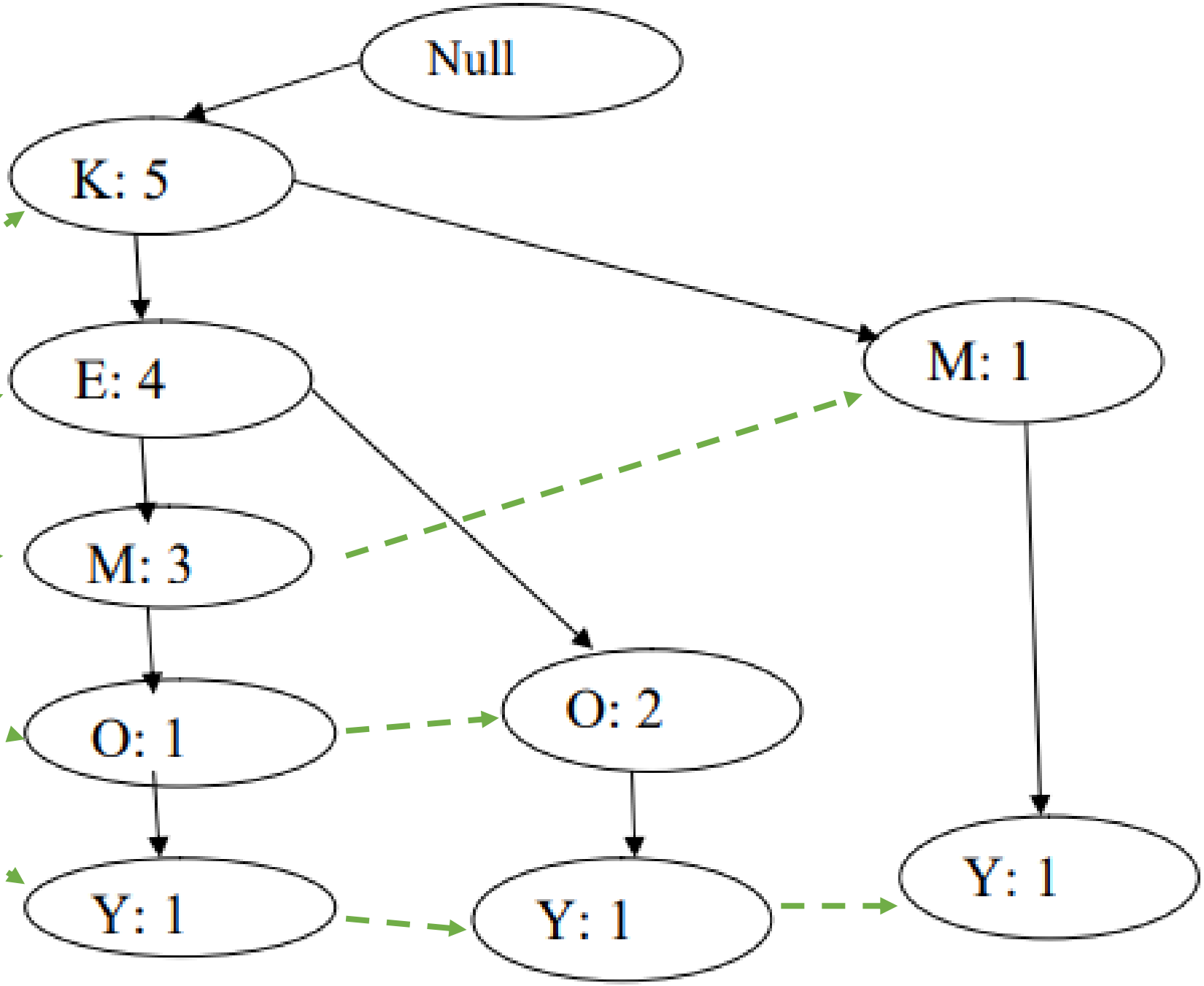
item	Support count
M	3
O	3
N	2
K	5
E	4
Y	3
D	1
A	1
U	1
C	2
I	1

item	Support count
M	3
O	3
K	5
E	4
Y	3

item	Support count
K	5
E	4
M	3
O	3
Y	3

TID	Itemset	Ordered itemset
100	{ M, O, N, K, E, Y }	{ K, E, M, O, Y }
200	{ D, O, N, K, E, Y }	{ K ,E ,O ,Y }
300	{ M, A, K, E }	{ K ,E ,M }
400	{ M, U, C, K, Y }	{ K ,M ,Y }
500	{ C, O, K, I, E }	{ K ,E ,O }

item	Support count	Node link
K	5	
E	4	
M	3	
O	3	
Y	3	



Item	Conditional Pattern Base	Conditional FP- tree	Frequent Pattern Generated
Y	{{KEMO:1}, {KEO:1}, {KM:1}}	<K:3>	{K,Y:3}
O	{{KEM:1}, {KE:2}}	<K:3, E:3>	{K,O:3}, {E,O:3}, {K,E,O:3}
M	{{KE:2}, {K:1}}	<K:3>	{K,M:3}
E	{K:4}	<K:4>	{K,E:4}
K	---	---	---

TID	Itemset
100	{ M, O, N, K, E, Y }
200	{ D, O, N, K, E, Y }
300	{ M, A, K, E }
400	{ M, U, C, K, Y }
500	{ C, O, K, I, E }

minsup=3
Using Apriori Algorithm

C1

item	Support count
M	3
O	3
N	2
K	5
E	4
Y	3
D	1
A	1
U	1
C	2
I	1

F1

item	Support count
M	3
O	3
K	5
E	4
Y	3

C2

item	Support count
MO	1
MK	3
ME	2
MY	2
OK	3
OE	3
OY	2
KE	4
KY	3
EY	2

F2

item	Support count
MK	3
OK	3
OE	3
KE	4
KY	3

C3

item	Support count
MKO	
MKE	
MKY	
OKE	3
OKY	
KEY	

F3

item	Support count
OKE	3

Alternative method for generating frequent Itemset

Several alternative methods have been developed to overcome the limitations and improve upon the efficiency of the Apriori algorithm.

Traversal of Itemset Lattice

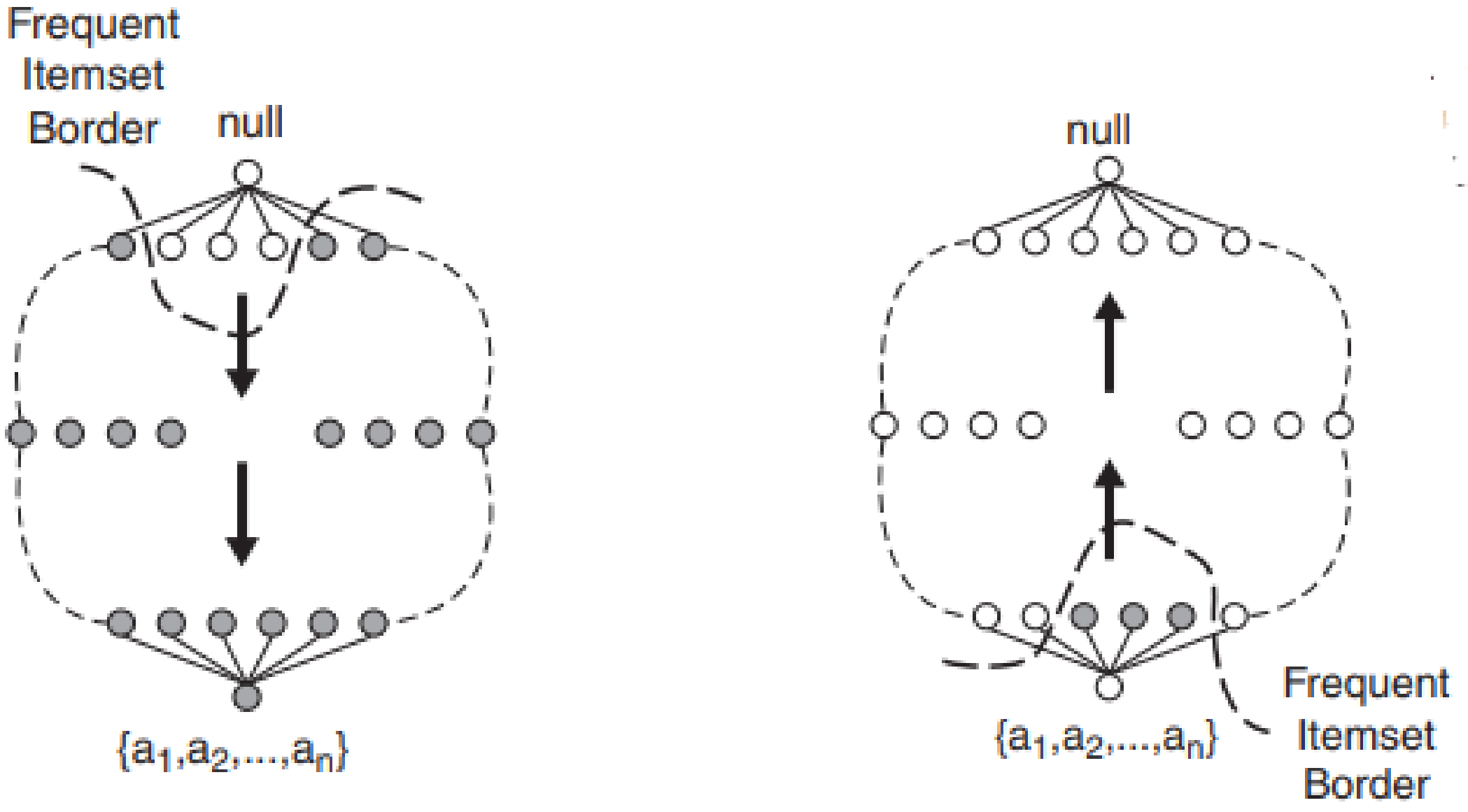
A search for frequent itemsets can be conceptually viewed as a traversal on the itemset lattice

Following are the search strategies:

General-to-Specific versus Specific-to-General

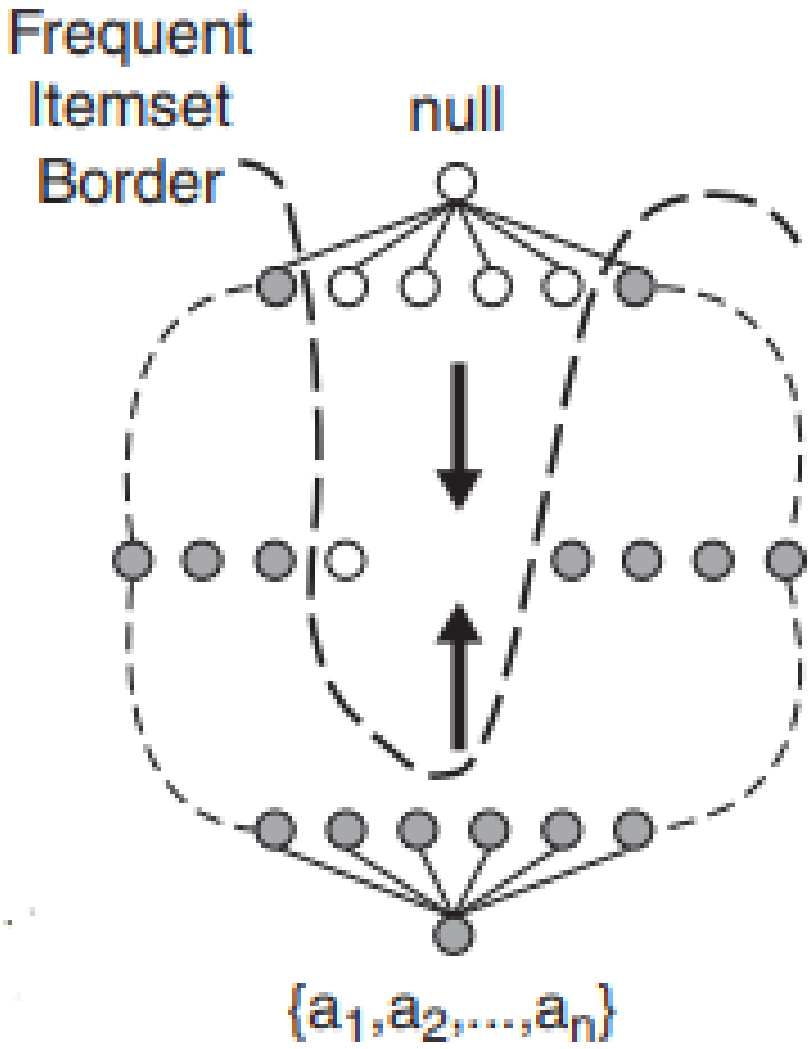
Equivalence Classes

Breadth-First versus Depth-First



(a) General-to-specific

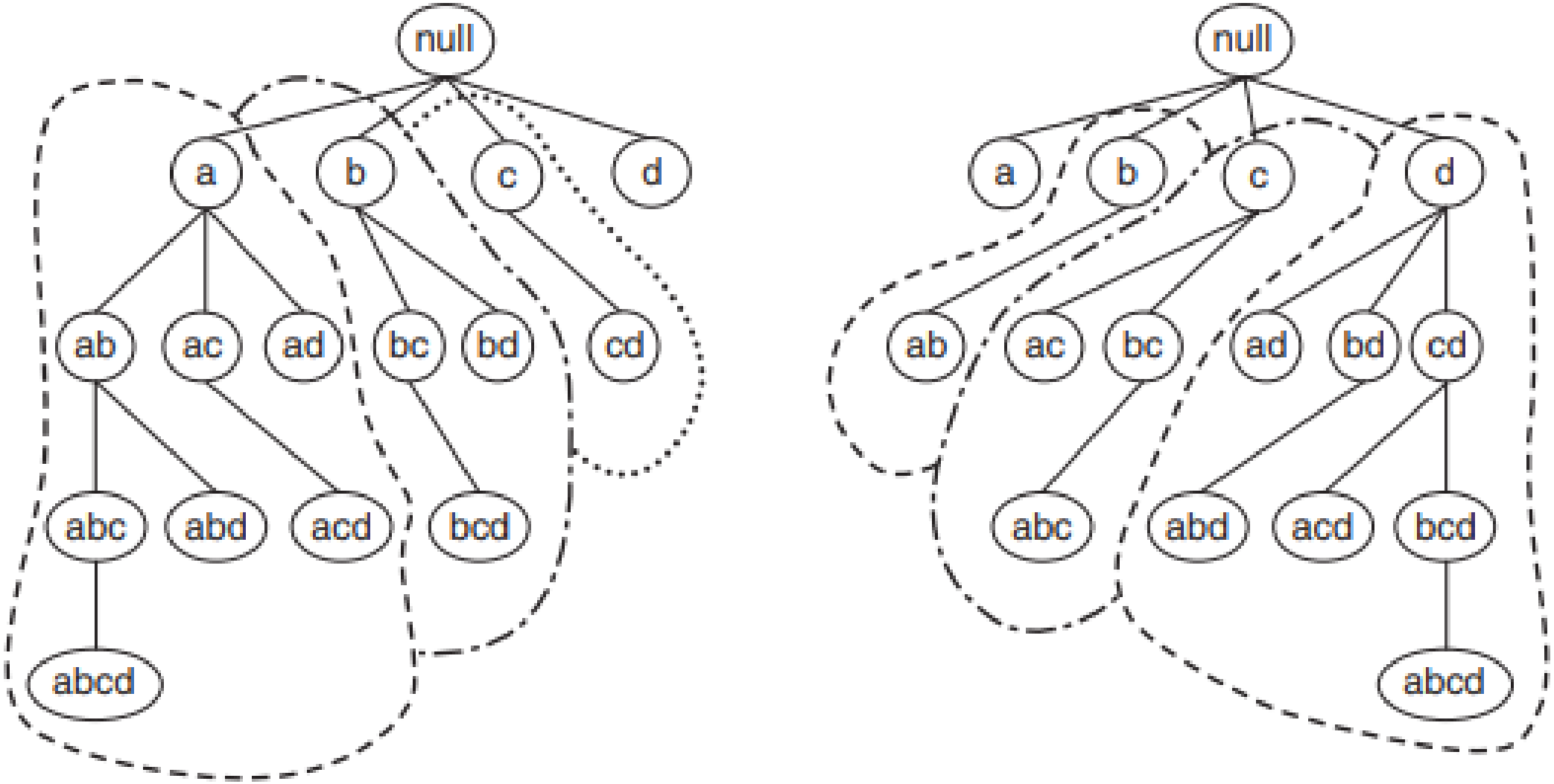
(b) Specific-to-general



(c) Bidirectional

Equivalence Classes

→ Partition the lattice into disjoint groups of nodes (or equivalence classes).

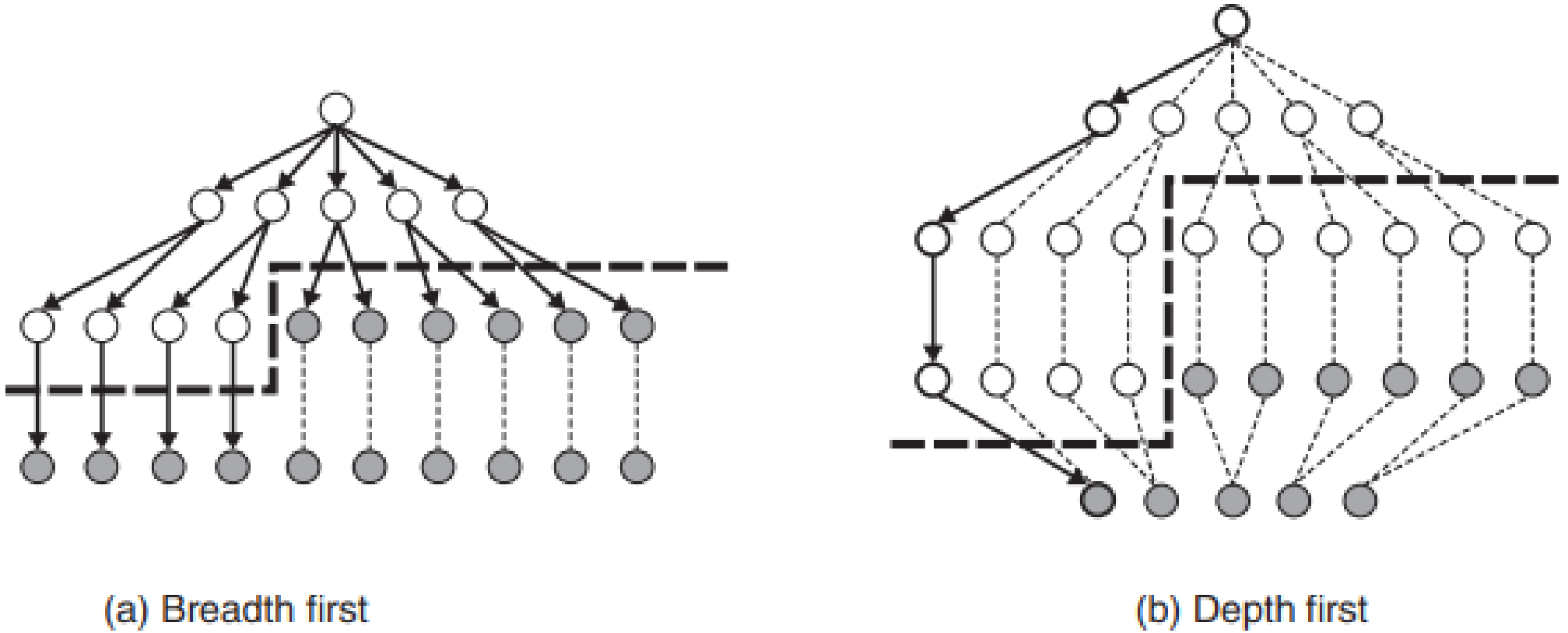


(a) Prefix tree.

(b) Suffix tree.

Figure 5.20. Equivalence classes based on the prefix and suffix labels of itemsets.

Breadth-First versus Depth-First



(a) Breadth first

(b) Depth first

Figure 5.21. Breadth-first and depth-first traversals.

Evaluation of Association Pattern

“One persons trash might be another persons treasure”

{Butter}→{Bread}

On the other hand,

{Diaphers}→{Beer}

Different types of association pattern evaluation criteria:

Support- Confidence framework

Interest factor

Correlation Analysis

IS Measure

Table 5.6. A 2-way contingency table for variables A and B.

	B	$\overline{B}$	
A	f_{11}	f_{10}	f_{1+}
$\overline{A}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	N

Suppose we are interested in analyzing the relationship between people who drink tea and coffee

Table 5.7. Beverage preferences among a group of 1000 people.

	$Coffee$	$\overline{Coffee}$	
Tea	150	50	200
$\overline{Tea}$	650	150	800
	800	200	1000

To evaluate the association rule

{Tea} →{Coffee}

It may appear that people who drink tea also tend to drink coffee because the rule's support (15%) and confidence (75%) values are reasonably high

The rule {Tea} → {Coffee} is therefore misleading despite its high confidence value.

Example 2

Table 5.8. Information about people who drink tea and people who use honey in their beverage.

	$Honey$	$\overline{Honey}$	
Tea	100	100	200
$\overline{Tea}$	20	780	800
	120	880	1000

If we evaluate the association rule {Tea} → {Honey}

Confidence value of this rule is merely 50%, which might be easily rejected

Interest Factor

The interest factor, which is also called as the “lift,” can be defined as follows:

$$I(A,B) = \frac{s(A,B)}{s(A) \times s(B)} = \frac{N f_{11}}{f_{1+} f_{+1}}.$$

$$I(A,B)\begin{cases} = 1, & \text{if } A \text{ and } B \text{ are independent;} \\ > 1, & \text{if } A \text{ and } B \text{ are positively related;} \\ < 1, & \text{if } A \text{ and } B \text{ are negatively related.} \end{cases}$$

For tea-coffee,

$$I = \frac{0.15}{0.2 \times 0.8} = 0.9375,$$

suggesting a slight negative relationship between tea drinkers and coffee drinkers

Table 5.7. Beverage preferences among a group of 1000 people.

	$Coffee$	$\overline{Coffee}$	
Tea	150	50	200
$\overline{Tea}$	650	150	800
	800	200	1000

For the tea-honey example,

$$I = \frac{0.1}{0.12 \times 0.2} = 4.1667,$$

Suggesting a strong positive relationship between people who drink tea and people who use honey in their beverage.

Limitations

Table 6.9. Contingency tables for the word pairs ($\{p,q\}$ and $\{r,s\}$.

	p	$\overline{p}$	
q	880	50	930
$\overline{q}$	50	20	70
	930	70	1000

	r	$\overline{r}$	
s	20	50	70
$\overline{s}$	50	880	930
	70	930	1000

P=data, q=mining,

r=compiler, s=mining

The interest factor for {p, q} is 1.02 and for {r, s} is 4.08.

Correlation Analysis

Correlation analysis is one of the most popular techniques for analyzing relationships between a pair of variables

$$\phi = \frac{f_{11}f_{00} - f_{01}f_{10}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}.$$

value of the ϕ-coefficient ranges from −1 to +1.

A correlation value of 0 means no relationship
while a value of +1 suggests a perfect positive relationship and
value of −1 suggests a perfect negative relationship.

tea and coffee drinkers example,

−0.0625, which is slightly less than 0.

On the other hand, the correlation between people who drink tea and people who use honey is 0.5847, suggesting a positive relationship.

Limitations

Table 6.9. Contingency tables for the word pairs ($\{p,q\}$ and $\{r,s\}$.

	p	$\overline{p}$	
q	880	50	930
$\overline{q}$	50	20	70
	930	70	1000

	r	$\overline{r}$	
s	20	50	70
$\overline{s}$	50	880	930
	70	930	1000

Although the words p and q appear together more often than r and s, their ϕ-coefficients are identical

ϕ(p, q) = ϕ(r, s)=0.232.

IS Measure

$$IS(A,B) = \sqrt{I(A,B) \times s(A,B)} = \frac{s(A,B)}{\sqrt{s(A)s(B)}} = \frac{f_{11}}{\sqrt{f_{1+}f_{+1}}}.$$

The value of IS thus varies from 0 to 1, where an IS value of 0 corresponds to no co-occurrence of the two variables, while an IS value of 1 denotes perfect relationship

For the tea-coffee example, the value of IS is equal to 0.375,

while the value of IS for the tea-honey example is 0.6455.

The IS measure thus gives a higher value for the {Tea}→ {Honey} rule than the {Tea}→ {Coffee} rule,

Table 5.9. Examples of objective measures for the itemset $\{A, B\}$.

Measure (Symbol)	Definition
Correlation (ϕ)	$\frac{Nf_{11}-f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$
Odds ratio (α)	$(f_{11}f_{00})/(f_{10}f_{01})$
Kappa (κ)	$\frac{Nf_{11}+Nf_{00}-f_{1+}f_{+1}-f_{0+}f_{+0}}{N^2-f_{1+}f_{+1}-f_{0+}f_{+0}}$
Interest (I)	$(Nf_{11})/(f_{1+}f_{+1})$
Cosine (IS)	$(f_{11})/(\sqrt{f_{1+}f_{+1}})$
Piatetsky-Shapiro (PS)	$\frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2}$
Collective strength (S)	$\frac{f_{11}+f_{00}}{f_{1+}f_{+1}+f_{0+}f_{+0}} \times \frac{N-f_{1+}f_{+1}-f_{0+}f_{+0}}{N-f_{11}-f_{00}}$
Jaccard (ζ)	$f_{11}/(f_{1+}+f_{+1}-f_{11})$
All-confidence (h)	$\min\left[\frac{f_{11}}{f_{1+}}, \frac{f_{11}}{f_{+1}}\right]$